

Decision-Focused Regime-Gated Reservoir Computing for Multi-Objective Portfolio Construction

Harsh Yadav

Supervisor: Prof. Peter Tiňo

School of Computer Science

College of Engineering and Physical Sciences

University of Birmingham

2025-26

Abstract

Portfolio optimization pipelines usually forecast returns and then optimize over those forecasts, which trains the model on prediction error rather than the quality of the decisions it makes. This dissertation studies an alternative in which training is done end-to-end on the risk-adjusted returns of the realized stream, gated by regimes discovered during training rather than statistically. The base underlying model is a Simple Cycle Reservoir, which is deterministically obtained and frozen, making the entire pipeline lightweight in both training and inference, making it cheap to scale. The central research question is whether such a simple reservoir model can outperform other popular methods like 1/N, Buy and hold SPY, Ledoit Wolf MV, LSTM, etc and whether decision-defined regimes carve the market differently than statistical regimes, and, if so, what underlying dimensions they vary upon.

I find that overall my methodology performs similarly to or worse than those baselines on the Sharpe ratio. However, I find that the learned gates carve the market differently. They focus more on recent market direction, while statistical gates focus more on volatility. Still, the true underlying driver of learned gates is undiscovered and may not be any of the general parameters used. Finally, increasing the number of experts does not significantly improve the obtained values. This study also shows that walk-forward hyperparameter selection does not transfer for this problem, so a configuration chosen on validation data can perform worse than the untuned default.

Acknowledgements

I would like to thank my supervisor, Professor Peter Tiňo, for his guidance throughout this project. Our regular meetings shaped the direction of the work, he helped me think through the ideas and the challenges I ran into, introduced me to concepts and techniques I may not otherwise have come across, and offered steady, practical advice from beginning to end. I am grateful for his time and patience.

I would also like to thank Professor Wendy Yanez Pazmino for her suggestions on the report, which helped improve how the work is presented.

Declaration

I certify that this dissertation is my own work. All ideas, analyses, and conclusions presented are my original work, developed under the supervision of Professor Peter Tiňo.

Generative AI tools were used for this project in the following ways. ChatGPT and Claude Opus were used to obtain and summarise research papers before reading them in depth. They were also used to generate LaTeX code for tasks such as tables and algorithms, where I provided the pseudocode and requirements and the tools converted them into the appropriate LaTeX structure. Grammarly was also used for grammar and spelling checks.

Finally, AI tools were used to assist with the initial report structure and boilerplate LaTeX code. Claude Code was used extensively during coding: I described the ideas and requirements in detail and reviewed the generated code. For areas involving important or non-generic code, I generated specific segments incrementally and incorporated them into the implementation after review. Claude's image-generation capabilities were also used, particularly for generating graphs and charts presented in this report.

Any code or written content produced with AI assistance has been reviewed, tested, and edited by me to ensure that it accurately reflects my own work and understanding of the project.

Abbreviations

CVaR	Conditional Value-at-Risk
ESN	Echo-State Network
HAC	Heteroskedasticity- and Autocorrelation-Consistent
HMM	Hidden Markov Model
MoE	Mixture of Experts
MSE	Mean Squared Error
MV	Mean-Variance
RC	Reservoir Computing
SCR	Simple Cycle Reservoir
SR	Sharpe Ratio
VIX	CBOE Volatility Index
AMI	Adjusted Mutual Information
API	Application programming interface

Contents

Abstract ii

Acknowledgements iii

Declaration iv

Abbreviations v

List of Figures viii

List of Tables ix

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Contribution . . . 2
- 1.3 Research questions . . . 2

2 Background and Related Work 3
- 2.1 Reservoir computing . . . 3
  - 2.1.1 Construction of the reservoir matrices . . . 4
  - 2.1.2 Universality of the cycle . . . 5
- 2.2 Regime models in finance . . . 5
- 2.3 Mixture-of-experts and decision-focused learning . . . 6

3 Methodology 7
- 3.1 The differentiable ledger and the decision loss . . . 7
- 3.2 The frozen reservoir . . . 8
- 3.3 The regime gate and the expert mixture . . . 9
- 3.4 Design principle: decision-focused learning . . . 10
- 3.5 Naive baselines and Statistical gates . . . 11
- 3.6 Evaluation protocol . . . 12

4 Data and Experimental Setup 14
- 4.1 Universes and calendars . . . 14
- 4.2 Market-level signals . . . 15
- 4.3 Reservoir and model hyperparameters . . . 15
- 4.4 Reproducibility . . . 16

5 Results **18**
5.1 What the regimes track . . . 18
5.2 The walk-forward ladder . . . 19
5.3 Capacity, selection and transfer . . . 20
5.4 The holdout . . . 23

6 Discussion and Limitations **26**
6.1 Why the carving differs . . . 26
6.2 Why it does not pay . . . 27
6.3 Methodological lessons . . . 27
6.4 Evaluation design . . . 28
6.5 Scope of the conclusions . . . 28
6.6 Model choices . . . 28

7 Conclusion and Future Work **30**
7.1 Future work . . . 30

A The second universe **33**

B Negative results and reversals **36**

C Folds and per-year results **38**

D GitLab link **40**

List of Figures

2.1 Minimum-complexity reservoir topologies 4

3.1 System architecture 8
3.2 Model architecture 10
3.3 Walk-forward and holdout protocol 13

4.1 Asset availability 16
4.2 Market-level signals 17

5.1 Regime timelines 19
5.2 Regime agreement 19
5.3 What each regime scheme tracks 20
5.4 Ladder comparison 22
5.5 Growth of £10,000 23
5.6 Capacity curve 24
5.7 Validation against test on the fine grid 25

A.1 Regime timelines, `etf10` 34
A.2 Regime agreement, `etf10` 34
A.3 Capacity curve, `etf10` 35

List of Tables

1.1 A example of lower MSE leading to worse decision outcome. 2

3.1 A more accurate forecast can make the worse decision. Model 1 has the lower mean squared error but earns less, because its small errors flip the ranking of the two assets, while Model 2's larger error preserves it. 11

4.1 The two universes. Every universe also carries a CASH column earning the risk-free rate. Two sector funds enter late and are handled by the availability matrix. . . . 15
4.2 The five market-level signals, their sources, and the publication lag applied before use. 15
4.3 Hyperparameters. Only the gate and experts are trained; the reservoir is frozen. The reported configuration is the default, not a searched winner. 17

5.1 The walk-forward ladder 21
5.2 Held-out verification of searched winners 21
5.3 Loss penalties select to zero 23
5.4 The holdout evaluation 24

B.1 The retired selection front 37
B.2 The price of cardinality 37
B.3 Mean-of-folds against the concatenated metric 37

C.1 Fold definitions 39
C.2 Per-year Sharpe, `sector11` 39
C.3 Per-year Sharpe, `etf10` 39

CHAPTER 1

Introduction

Given a particular universe of assets, deciding the optimal way to split money across them is one of the oldest problems in quantitative finance, and it has an extremely large and ever-growing literature. This dissertation approaches it from a particular angle: instead of a large, complex model, I use a simple deterministic Simple Cycle reservoir, combined with the idea of training a portfolio model directly on the quality of the decisions it makes. On top of that, I ask about market regimes and try to identify them as they evolve, rather than using a specific statistical method.

1.1 Motivation

The prevalent approach in a data driven portfolio construction from a top level view is a two stage pipeline: predict each assets future returns and hand those returns to an optimizer that changes the models weights/parameters. The central model is thus often trained to minimize the optimizer's prediction error, most often mean squared error with some other loss terms. This is convenient but it optimises a proxy rather than something of higher importance. Prediction accuracy and decision quality are different objectives, they can disagree at times, a model with lower MSE can still make worse allocation, mainly because MSE penalises every mistake in proportion to its size while actual allocation only cares on getting the relevant comparison right. Consider the case in 3.1 with only 2 assets in our universe, and we have two model, the corresponding MSE are also noted.

If the goal was to choose one asset, then choosing lower MSE leads us to Asset B, which will yield worse earnings. If the goal was to generate a full portfolio, the effect is reduced but not removed. The model will generally spread its weight in proportion to its predicted returns. Model 1 predicts $(A = 2\%, B = 3\%)$, so it puts about 40% in A and 60% in B, earning $0.4(4\%) + 0.6(1\%) = 2.2\%$. Model 2 predicts $(A = 7\%, B = -1\%)$, so it concentrates on A and earns about 4%. The lower-MSE model still loses because it ranks B above A, but the margin shrinks from 3% in the single-asset case to about 1.8% here. In conclusion, ranking errors hurt portfolio optimization, and MSE can create worse ranking orders. Thus, many models are imperfect [1], which motivates me to train end-to-end on the decision itself, measuring success by risk-adjusted realized returns rather than any per-period forecast error [2].

The second idea is a long standing one that markets move through distinct *regimes* [3], and that the allocation should adapt to which regime we current are in. The simplest picture is a bullish market vs a bearish one. Most work build these regimes concrete with a statistical rule. The two most common being hidden Markov models [3] and thresholds on a volatility index such as the VIX. There is a vital issue to such regime generation, we assume that we know what a regime is and what shape it takes. If the structure that actually matters for a decision is something a Gaussian state model or a volatility bucket cannot express, imposing one of those forms in advance

	Asset A	Asset B	MSE	buys	earns
Actual	4%	1%	—	—	—
Model 1	2%	3%	4.0	B	1%
Model 2	7%	−1%	6.5	A	4%

Table 1.1: A example of lower MSE leading to worse decision outcome.

may hide it. A natural alternative is to let the decision objective discover the regimes for itself, so that the notion of a regime is defined by what helps the allocation rather than by a modeling assumption made beforehand.

Finally, I wanted a model that's cheap to train and run, rather than a large, complex one, and to see how far I could take it. I also wanted my work to be reproducible, so I wanted it to be as deterministic as possible. I ended up choosing an echo state network, specifically a Simple Cycle Reservoir whose recurrent structure is fixed and deterministic [4]. On top of the frozen encoder sits a mixture-of-experts allocator and a gating network that routes among them [5]. All of these are simple linear readouts, and only they are trained, in the reservoir we only tune its hyperparameters.

1.2 Contribution

The contribution of this dissertation is the unique blend rather than any single ingredient. I build a portfolio optimization model in which trainable gates discover regimes on the states of a *frozen* reservoir, a set of expert heads propose various portfolios, and the entire system is trained end-to-end on the excess Sharpe ratio of the realized return stream, accounting for transaction fees. Because the ledger that turns proposed portfolios into metrics, including Sharpe ratio, is differentiable, the decision objective backpropagates directly to the model, and no component forecasts returns. This leads to a research question: do regimes defined by the decision objective carve the market differently than regimes defined statistically?

My answer has three central parts. First, the decision defined regimes do carve the market differently, and I was able to generate some idea of the underlying mechanism: the learned gate has a higher significance on recent market direction while the statistical gates focus more on volatility, so the two scheme read orthogonal axis and agree no more than by chance. Second this difference does not directly translate into performance. Across both universes with walk forward evaluation, no learned strategy significantly beats a naive baseline. Third, and of independent interest, I find that hyperparameter selection does not transfer on this problem: a configuration chosen on validation data can perform worse out of sample than the untuned default, which is why the frozen configuration I report is the default rather than a searched winner. A single, fire walled holdout, touched only once after the configuration was frozen, confirms the null on data the model had never seen.

1.3 Research questions

The study is organised around three main questions.

RQ1: Do regimes discovered by the decision objective carve the market differently from statistically defined regimes (a filtered hidden Markov model and VIX terciles)? If so, what does each scheme actually track?

RQ2: Does this decision-defined regime gating *pay*, that is, does it produce risk-adjusted performance that beats naive and mean-variance baselines out of sample.

RQ3: Does adding capacity or information help, specifically a larger number of experts, a trained temporal encoder in place of the frozen reservoir, and market-level signals fed to the gate?

CHAPTER 2

Background and Related Work

This chapter reviews the three key literature I draw on: reservoir computing, regime models in finance, and mixtures of experts together with decision-focused learning. It also presents, in full, the reservoir construction the dissertation uses, since that construction comes directly from the literature rather than from me.

2.1 Reservoir computing

Learning from a time series requires a model that carries information forward in time. Recurrent neural networks do this by passing a state from one step to the next, but training them by propagating gradients back through time is expensive and on short, noisy samples it is also data-hungry. Reservoir computing (RC) is a family of models built around a simple alternative: fix the recurrent part and train only the map that reads it [6, 7]. The fixed recurrent network is called the *reservoir*; driven by the input stream it acts as a rich, nonlinear memory, and a simple readout, extracts from its state whatever the task needs.

The best known RC model is the Echo State Network (ESN) [6]. With input $s(t)$, reservoir state $x(t)$ and readout output $y(t)$, its dynamics and readout are

$$x(t+1) = f\big(Wins(t+1) + Wx(t) + b\big), \qquad y(t+1) = U\,x(t+1), \tag{2.1}$$

where f is a sigmoid-type activation, typically tanh. Only U is trained; W and Win are drawn at random and then left alone. For the state to be a usable memory it must be an "echo" of the input history rather than of its own arbitrary starting point, the *echo state property*, which is usually arranged by scaling W so that its spectral radius is below one, although that scaling alone is neither necessary nor sufficient [4]. Our system uses a leaky variant of Eq. 2.1, given later as Eq. 3.5, in which the new state is blended with the old one to lengthen memory.

The standard ESN recipe attracts a standing criticism, set out in [4]: the reservoir is a randomised black box, its properties are poorly understood, getting a good one takes repeated trials and some luck, and the spectral radius is a weak tool for controlling what the dynamics actually do. Prof. Rodan and Prof. Tiňo therefore asked how much of the randomness and connectivity is necessary at all, and answered: almost none. They study three deterministic templates, a delay line (DLR), a delay line with feedback (DLRB), and the *Simple Cycle Reservoir* (SCR), in which the units form a single ring, every connection in the ring carries the same weight r, and every input connection carries the same absolute weight v (Figure 2.1). The whole reservoir is then specified by two numbers. Across a wide range of standard benchmarks, NARMA systems, the Santa Fe laser series, the Hénon map, a nonlinear communication channel, radar and sunspot series, and a speech task, these minimal reservoirs perform comparably to the fully random ESN; on the NARMA family the SCR is the best or statistically not worse than the best.

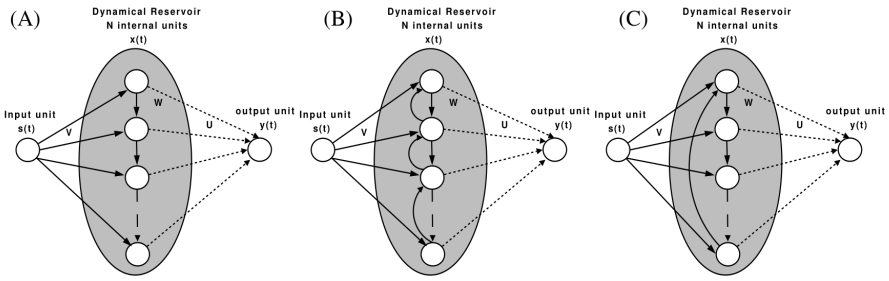


Figure 2.1: The three minimum-complexity reservoir templates :(A) the delay line reservoir, (B) the delay line with feedback, and (C) the Simple Cycle Reservoir used in this dissertation, in which the units form a single ring with one shared weight. Reproduced from Fig. 2 of [4].

One residual source of randomness remains in their construction: the signs of the input weights.Imposing any *periodic* sign pattern hurts performance, but any *aperiodic* pattern works, and it can be generated deterministically, for example by thresholding the decimal digits of π, digits below five giving a negative sign, or from the digits of e, or from a logistic-map trajectory [4]. With that last piece the reservoir contains no randomness at all.

They also characterise what the simplification costs in memory. The short-term memory capacity MC of a reservoir sums, over all delays k, the squared correlation between the input delayed by k steps and the network's best reconstruction of it. A classical result of Jaeger bounds $MC \leq N$ for any N-unit network. For the linear SCR the paper proves the capacity exactly:

$$MC \;=\; N - \left(1 - r^{2N}\right), \tag{2.2}$$

so that choosing $r = (1-\epsilon)^{1/2N}$ places the capacity within any desired ϵ of the proven optimum N. The ring loses essentially nothing.

2.1.1 Construction of the reservoir matrices

This dissertation adopts the SCR construction exactly, writing the cycle weight as ρ because, for a ring, the shared weight and the spectral radius coincide. Both matrices are fixed and fully deterministic: nothing here is trained or drawn at random.

The recurrent matrix W - a single cycle. The N units are wired into one ring with a single shared weight equal to the spectral radius ρ. Unit i receives from unit $i-1$, and unit 0 receives from the last unit, closing the loop:

$$W_{ij} = \begin{cases} \rho & \text{if } j \equiv (i-1) \bmod N, \\ 0 & \text{otherwise,} \end{cases} \qquad W = \rho \begin{pmatrix} 0 & 0 & \cdots & 0 & 1 \\ 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{pmatrix}. \tag{2.3}$$

The nonzeros lie on the sub-diagonal plus the top right corner (the wrap-around). The eigenvalues of such an N-cycle are ρ times the N-th roots of unity, so every eigenvalue has modulus exactly $|\rho|$ and the spectral radius *is* ρ by construction leading to no rescaling and no eigenvalue solve, we just have this hyperparameter ρ.

The input matrix Every input connection shares a single magnitude v (the calibrated input scale); only its sign and the unit it feeds vary.

$$W_{\text{in}} \;=\; v\, D\, C, \tag{2.4}$$

where the sign diagonal $D \in \mathbb{R}^{N\times N}$ and the cyclic assignment $C \in \{0,1\}^{N\times M}$ are

$$D = \begin{pmatrix} s_0 & & & \\ & s_1 & & \\ & & \ddots & \\ & & & s_{N-1} \end{pmatrix}, \qquad C = \underbrace{\begin{pmatrix} I_M \\ I_M \\ \vdots \\ I_M \end{pmatrix}}_{N/M \text{ copies}}, \qquad s_i = \begin{cases} +1 & d_i \geq 5, \\ -1 & d_i < 5, \end{cases} \tag{2.5}$$

with I_M the $M \times M$ identity and d_i the i-th decimal digit of π [4], computed from Machin's formula $\pi = 16 \arctan \frac{1}{5} - 4 \arctan \frac{1}{239}$ rather than transcribed. So unit i listens to input $i \bmod M$ with a fixed $\pm$ sign, and each asset drives an equal share of the reservoir.

2.1.2 Universality of the cycle

A fair worry about an architecture so constrained is expressive power: does the ring give something up that a fully random reservoir keeps? For the reservoir class as a whole the answer was already known to be reassuring [8]: reservoir systems are universal approximators of time-invariant fading-memory filters, and universality holds even with *linear* state dynamics provided the trainable readout is rich enough. What remained open was whether the restricted cycle(SCR) retains this power. Li, Fong and Tiňo [9] settle it: simple cycle reservoirs, with the ring weight taken complex and input couplings restricted to $\{\pm 1, \pm i\}$, can approximate any unrestricted linear reservoir system with continuous readout to arbitrary accuracy, and therefore any time-invariant fading-memory filter over uniformly bounded inputs.

Two remarks connect these results to the design of this study. First, they are what licenses freezing the encoder: the SCR is not an ad hoc simplification but a construction whose memory is characterised exactly and whose expressive ceiling is known, so the interesting questions move to where the training data are scarcest, the readout and the objective. Second, universality is a statement about sufficiently rich readouts on the frozen state. This dissertation deliberately keeps its readouts *linear*, trading the expressive ceiling for trainability on a few hundred decisions per fold, and that trade has exact consequences: Chapter 6 shows that one family of market features, the even functions of returns that volatility belongs to, is provably out of a linear readout's reach on this state. The same structure that makes the reservoir analysable makes its limits analysable too.

2.2 Regime models in finance

The idea that markets move between a small number of conditions is old and well developed. One of this popular statistical form is the Markov switching model of Hamilton [3], in which an unobserved discrete state follows a Markov chain and the observed series changes its dynamics with the state. Applied to US output, the two recovered states aligned with expansions and recessions. The machinery for such models, filtering, smoothing and EM fitting, is that of hidden Markov models [10]. One key distinction is that the *filtered* state probability uses only observations up to the current day, while the *smoothed* probability conditions on the whole sample, future included. Only the filtered quantity is legitimate in a backtest, and Section 3.5 implements the HMM gate with a causal filter for exactly this reason.

Ang and Bekaert [11] document a high-volatility, high-correlation regime in international equity returns, the conditions in which diversification helps least arriving together, and study its consequences for allocation. Guidolin and Timmermann [12] estimate multivariate regime-switching models for stock and bond returns and find the optimal allocation changes substantially with the regime. Alongside these estimated models sits a simpler tradition that needs no fitting at all: classifying the day by the level of the VIX index. The VIX-tercile gate of Section 3.5 encodes that heuristic in causal form.

What every scheme in this section shares is the order of operations: the regimes are defined first, from the statistics of returns or from a chosen index, and decisions follow. Whether that order is the right one is precisely what this dissertation asks.

2.3 Mixture-of-experts and decision-focused learning

The mixture of experts of Jacobs, Jordan, Nowlan and Hinton [5] is the architecture this dissertation uses to make regimes learnable. Several small networks i.e the experts, propose outputs, and a gating network assigns each input a soft weighting over them. Trained together, the gate learns to split the input space and each expert learns to specialise on its share. The regime interpretation is immediate: the gate's weighting *is* a soft regime label, and the experts are per-regime policies. Chapter 3 uses exactly this, with the reservoir state as the gate's input and portfolios as the experts' outputs.

The second ingredient is the objective. The classical portfolio pipeline predicts first and optimises second: estimate expected returns and covariances, then hand the estimates to a mean-variance optimiser. Its known weakness is estimation error, which the optimiser amplifies. The shrinkage covariance estimator of Ledoit and Wolf [13] is the standard remedy and serves as this study's strongest classical baseline. But a deeper problem remains even with good estimates which is that the forecaster is trained on prediction error, which is only a proxy for decision quality, and the two can disagree, as the worked example of Section 3.4 shows. The decision-focused learning literature addresses this by training the predictor under the loss induced by the downstream decision, the "Smart Predict-then-Optimize" framework of Elmachtoub and Grigas [1]. This dissertation takes the idea one step further: it removes the prediction target altogether and trains the portfolio map end to end on the realised risk-adjusted return itself, the excess Sharpe ratio [2], made differentiable by writing the accounting ledger in the same tensor framework as the model.

CHAPTER 3

Methodology

This chapter sets out the methodology used in this project in the order I implement and run it. I begin with the design principle, which is the base that affects every later choice; namely, the portfolio ledger that runs suggested weights on any provided time frame. It is model-agnostic and is used to run every model in this project. I then download all the data, preprocess it, and set aside the holdout set. Then I train a Simple Cycle Reservoir (SCR) on the log returns and cache it for future use. I then provide these saved states to K experts and a gating network, which are linearly joined. Furthermore, the model is trained on the quality of its decisions rather than the accuracy of any forecast. That quality becomes our loss function, creating a cycle that trains our experts and the gating network. The rest of the chapter describes the statistical gates I compare against and the protocol under which I evaluate every arm.

3.1 The differentiable ledger and the decision loss

Given a weight schedule, the ledger plays the portfolio day by day and returns key metrics like Sharpe ratio, turnover, max drawdown, etc. One key point is that the weight schedule is weekly, i.e we only rebalance once per week. I do this because daily rebalances in our architecture and data would mostly chase noise, and they would also lead to extremely high net transaction costs. Returns are still calculated daily because the return values we cache are daily. The exact structure of the weight schedule is as follows

$$\text{targets} \in \mathbb{R}^{J \times N}, \tag{3.1}$$

where

- the J rows are the weekly decision points ;
- the N columns are the assets, the risky universe plus cash.

Each row is a target portfolio w_t , thus it's important for each of its elements to be non-negative and for all elements to sum to 1. Signals are formed on the last trading day of each week and executed on the following day, so the lag between decision and execution lives outside the ledger. Transaction costs are charged on the traded notional at five basis points, cash leg included, so that turnover is paid for at every rebalance. Within a holding period the daily ledger is closed form, thus for each asset all future returns are known thus we can directly calculate for that independently of other. Since this is same for all we can use direct dot product to replace a day-by-day loop making implementation faster. Finally the ledger is written in the same tensor library as the model, so gradients flow from every daily return back to the weights that produced it making training easier. The entire data processing and portfolio ledger architecture can be visually seen in figure 3.1

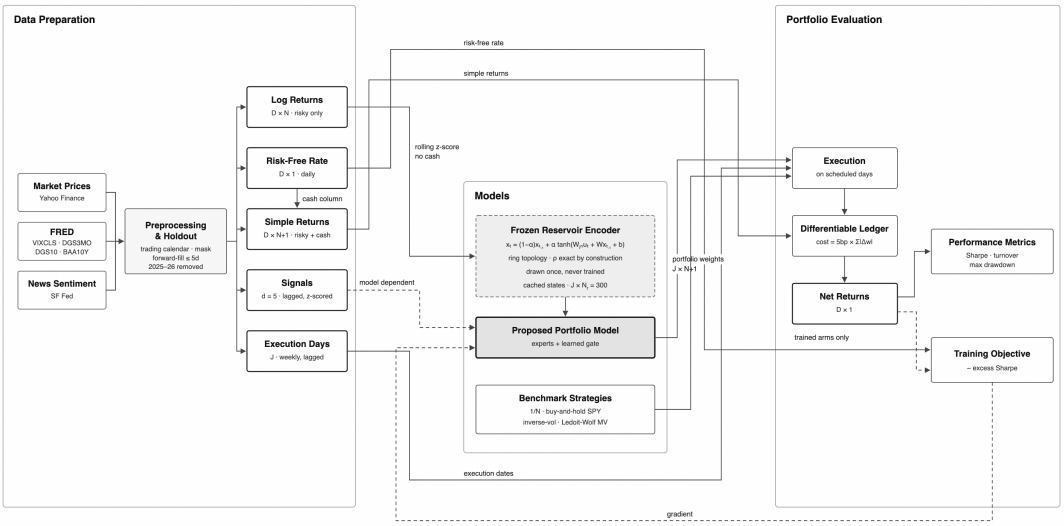


Figure 3.1: The pipeline for input and evaluation harness. We prepare the data and daily log returns drive a frozen Simple Cycle Reservoir whose state x_t is cached once. Any model can produce a portfolio which is played through a differentiable ledger with transaction costs, and its excess Sharpe ratio defines the loss.

Let r_t^{net} denote the daily net return of the blended portfolio and r_t^f the risk-free rate that cash earns over the same day. The primary training signal is the annualised excess Sharpe ratio of the realised stream,

$$\text{SR} = \sqrt{252}\,\frac{\text{mean}_t\left(r_t^{\text{net}} - r_t^f\right)}{\sqrt{\text{var}_t\left(r_t^{\text{net}} - r_t^f\right) + \varepsilon}}, \tag{3.2}$$

with a small constant ε that keeps the ratio finite when the excess stream is near zero. This is needed for cash, where both the numerator and denominator would be zero, or, in practice, an arbitrarily small number, which would give an unpredictable Sharpe; adding this makes its Sharpe 0, so it does not accidentally become a money haven. For our project, I set it to 10^{-6}. The same ε appears in the reporting metric, so the quantity optimized is the quantity reported. The training loss is the negative Sharpe ratio with two optional penalties,

$$\mathcal{L} = -\,\text{SR} + \lambda_{\text{cvar}}\ \text{CVaR}_{95} + \lambda_{\text{bal}}\ \text{CV}^2 \tag{3.3}$$

where the first penalty is meant to prevent the worst outcomes(i.e. days with huge loss) through the conditional value-at-risk of the excess stream and the second discourages expert collapse using square of coefficient of variation(CV). This is calculated as follows:

$$\text{CV}^2(I) = \frac{\sigma_I^2}{\bar{I}^{\,2}}, \qquad \bar{I} = \frac{1}{K}\sum_{k=1}^{K} I_k, \qquad \sigma_I^2 = \frac{1}{K}\sum_{k=1}^{K}\left(I_k - \bar{I}\right)^2, \tag{3.4}$$

where $I_k = \dfrac{1}{T}\sum_{t=1}^{T}\pi_{t,k}$ is the mean weight of that expert gate over the T decisions. The load-balancing term is identically zero at $K = 1$. Both weights are searchable and default to zero, so the plain objective is $-\,\text{SR}$. The Sharpe ratio couples every day,i.e its value can be calculated on any number of days, and its value on all days in the fold is the value we optimize on rather than on mini-batches.

3.2 The frozen reservoir

At the core of our project is a frozen Simple Cycle Reservoir. This is a fixed and deterministic recurrent system driven by the input log returns, from which only a lightweight readout is trained.

The central driving equation to obtain the states of the reservoir is as follows:

$$x_t = (1-\alpha)\, x_{t-1} + \alpha \tanh\big(W x_{t-1} + v\, W_{\text{in}} u_t\big), \tag{3.5}$$

where

$$u_t \;=\; \begin{pmatrix} r_{1,t} \\ r_{2,t} \\ \vdots \\ r_{M,t} \end{pmatrix} \;=\; \begin{pmatrix} \ln\big(P_{1,t}/P_{1,t-1}\big) \\ \ln\big(P_{2,t}/P_{2,t-1}\big) \\ \vdots \\ \ln\big(P_{M,t}/P_{M,t-1}\big) \end{pmatrix} \;\in\; \mathbb{R}^M. \tag{3.6}$$

- $x_t \in \mathbb{R}^N$ is the reservoir state at time t;
- $\alpha \in (0, 1]$ is the leak rate (smaller $\alpha \Rightarrow$ longer memory);
- $W \in \mathbb{R}^{N\times N}$ is the fixed recurrent matrix (the Simple Cycle ring);
- $W_{\text{in}} \in \mathbb{R}^{N\times M}$ is the fixed input matrix;
- $P_{i,j}$ is the adjusted closing price of asset i on day t

Here we have two matrix, both matrices follow the Simple Cycle Reservoir construction given in Section 2.1.1 exactly (Eqs. 2.3–2.4); nothing in them is trained or drawn at random.

Three implementation choices matter for later chapters. First, the input scale v is not set by hand but calibrated so that the standard deviation of the pre-activation $Wx_{t-1} + vW_{\text{in}}u_t$ reaches a target of about 0.7, which places the activations in the curved part of tanh without saturating them; the calibration uses only the first 60% of the sample and so never sees the evaluation period. Second, the first 250 trading days of each driven series are discarded as a washout, so the arbitrary choice $x_0 = 0$ has decayed away before any state is used. Third, because the reservoir is never trained, its states can be computed once over the whole sample and cached; a training run then reads precomputed states instead of driving the recurrence, which is what makes the roughly one thousand fits in the study affordable on a laptop.

3.3 The regime gate and the expert mixture

The trained part of the model is a mixture of experts [5]. At each weekly decision day t the reservoir supplies the state x_t, and two kinds of readout act on it. The first is the experts, they are defined as follows

$$w_t^{(k)} = \text{softmax}\big(W_k x_t + m_t\big), \tag{3.7}$$

in which m_t is an additive mask whose entries are 0 for assets that are tradeable on day t and a large negative constant otherwise, so that the softmax weight on an unavailable asset underflows to exactly zero while cash is never masked. This is required because two assets in the sector universe were added long after its inception.

The second is the gating network that produces a softmax response where the values are how likely the gating network believe the corresponding expert to be.

$$\pi_t = \text{softmax}\Big(\big(W_g x_t + b_g + \sigma_g\, \varepsilon_t\big)/\tau\Big), \qquad \varepsilon_t \sim \mathcal{N}(0, I_K), \tag{3.8}$$

where $W_g x_t + b_g$ are the gate logits, τ is the temperature that controls how sharply the membership concentrates, and $\sigma_g\, \varepsilon_t$ is Gaussian exploration noise added to the logits *during training only* ($\sigma_g = 0$ at evaluation). The noise standard deviation is annealed linearly to zero,

$$\sigma_g(e) = \sigma_0\, \max\Big(0,\; 1 - \frac{e}{P}\Big), \qquad \sigma_0 = 0.3, \quad P = 2\,\text{patience}, \tag{3.9}$$

so the routing explores early before the experts have differentiated, so no expert is starved and the gate commits once the noise has decayed to zero at epoch P.

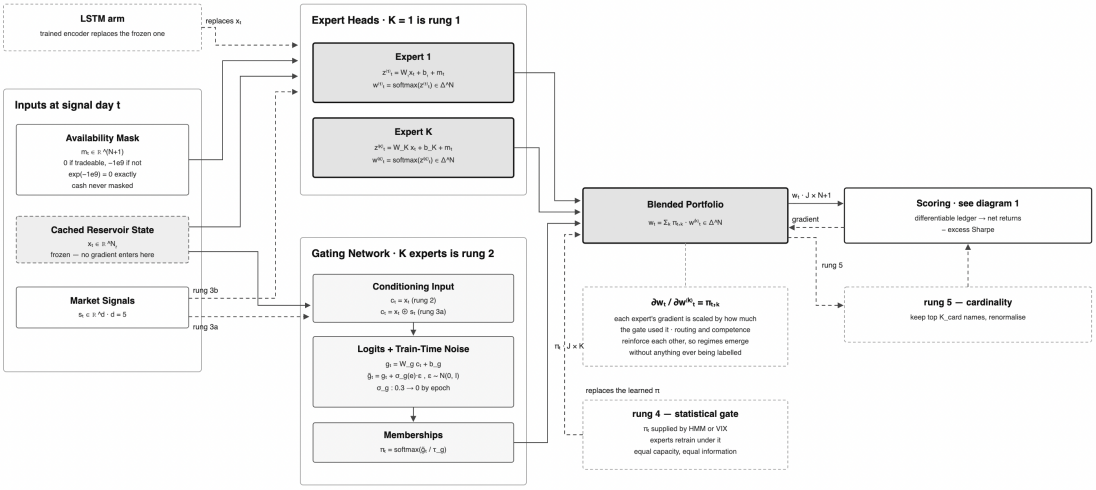


Figure 3.2: The reservoir inputs (and optionally market sentiment) are provided both of gates and experts and are combined together to generate final model's portfolio suggestion.

The final target portfolio is the membership-weighted blend of the experts,

$$w_t = \sum_{k=1}^{K} \pi_{t,k}\, w_t^{(k)}. \tag{3.10}$$

The number of experts K is a configuration parameter rather than a fixed constant. At $K = 1$ the mixture reduces provably to a single trained head and the gate is inert, which gives a clean baseline within the same code. Market-level signals may optionally be concatenated to the state before the gate, the experts, or both; keeping the signals market-level, and separate from the per-asset state, is what lets us treat them as an input the gate *may* use rather than as a definition of a regime. This entire architecture can be seen visually at 3.2.

3.4 Design principle: decision-focused learning

A conventional pipeline forecasts each asset's return and then hands those forecasts to a separate optimiser. The forecaster is trained to minimise a prediction error such as the mean squared error, which is a proxy for the thing we actually care about. The proxy and the objective can disagree. A model with the lower prediction error can still make the worse allocation. Methods like minimizing MSE treats every asset's error as equally important and symmetric. But for the allocation, an error in a high-weight asset matters far more than in a near-zero-weight one, An overconfident forecast that preserves the correct ranking is harmless to the decision. Training on the surrogate optimises the wrong quantity whenever the two come apart, and they come apart precisely because the model is imperfect [1]. To understand this fully consider the following example Two assets, A and B, will realise returns of 4% and 1% respectively. Two models forecast these returns, and a simple strategy acts on each forecast by buying the single asset it rates highest; the strategy then earns that asset's *actual* return. Table 3.1 shows the outcome.

The mean squared errors are

$$\text{MSE}_1 = \tfrac{1}{2}\big[(2-4)^2 + (3-1)^2\big] = 4.0, \qquad \text{MSE}_2 = \tfrac{1}{2}\big[(7-4)^2 + (-1-1)^2\big] = 6.5, \tag{3.11}$$

so by prediction error Model 1 is the better forecaster ($4.0 < 6.5$). Yet it is the worse investor: it rates B above A and earns 1%, whereas Model 2 rates A above B and earns 4%.

I therefore train end to end on the decision itself. The loss is the risk-adjusted quality of the entire realised return stream, measured by the excess Sharpe ratio [2], and the gradient of that loss

	Asset A	Asset B	MSE	buys	earns
Actual	4%	1%	—	—	—
Model 1	2%	3%	4.0	B	1%
Model 2	7%	−1%	6.5	A	4%

Table 3.1: A more accurate forecast can make the worse decision. Model 1 has the lower mean squared error but earns less, because its small errors flip the ranking of the two assets, while Model 2's larger error preserves it.

Algorithm 1 Decision-focused training of one mixture on one fold

input: cached states x, train/validation packs, experts K, loss weights, seed
initialise gate $\{W_g, b_g\}$ and experts $\{H_k\}$ from the seed
best $\leftarrow -\infty$
for epoch $= 1, 2, \ldots$ until early stopping **do**
  $\pi_t \leftarrow \text{gate}(x_t)$ with annealed logit noise {Eq. 3.8}
  $w_t \leftarrow \sum_k \pi_{t,k}\, w_t^{(k)}$ over the training fold {Eq. 3.10}
  run the differentiable ledger to obtain r_t^{net}
  $\mathcal{L} \leftarrow -\,\text{SR} + \lambda_{\text{cvar}}\,\text{CVaR}_{95} + \lambda_{\text{bal}}\,\text{CV}^2$ {Eq. 3.3}
  backpropagate $\mathcal{L}$ and update the gate and experts only
  evaluate the noise-free Sharpe on the validation fold
  if validation Sharpe improves **then**
    best $\leftarrow$ current parameters
  end if
end for
return the parameters at best

flows back through the portfolio and into the parameters that produced it. No part of the system forecasts a return. This choice is what makes the accounting ledger of Section 3.1 a differentiable object rather than a reporting step: the ledger *is* the forward pass of the loss. Additionally since we are optimizing Sharpe ratio this also accounts for volatility as well instead of just profits.

The same principle motivates the research question the study is built to answer. If regimes are useful, they should be discovered by the decision objective, not imposed beforehand by a statistical rule. The falsifiable question is whether decision-defined regimes carve the market differently from, and outperform, statistically defined regimes at equal capacity and equal information. Sections 3.3 and 3.5 construct the two sides of that comparison so that they differ only in how the regimes are formed.

3.5 Naive baselines and Statistical gates

Naive baselines: Four allocation rules with no regime structure set the humility line: an equal-weight portfolio over the active universe, an inverse-volatility portfolio on a 63-day window, buy-and-hold on the market index, and a Ledoit-Wolf shrinkage mean-variance portfolio [13] that holds risky assets under a budget constraint and leaves the residual in cash. Every baseline is run through the same ledger, the same cost model and the same evaluation windows as the learned arms.

Filtered hidden Markov gate: A Gaussian hidden Markov model [3, 10] is fitted, on the training window only, to two market observables: the equal-weight excess return of the active universe and its trailing 21-day realised volatility. Each hidden state k emits observations from a diagonal Gaussian $\mathcal{N}(\mu_k, \Sigma_k)$, and a transition matrix A governs how states follow one another. The membership used at day t is the *filtered* posterior $p(\text{state}_t \mid o_{1:t})$, computed by a forward recursion in log space,

$$\log \alpha_t(j) = \text{logsumexp}_i \left(\log \alpha_{t-1}(i) + \log A_{ij} \right) + \log \mathcal{N}(o_t;\, \mu_j, \Sigma_j), \qquad \pi_t = \frac{\alpha_t}{\sum_j \alpha_t(j)}. \tag{3.12}$$

The forward filter conditions only on observations up to and including day t. The library routine returns *smoothed* posteriors from the forward-backward algorithm, which condition on the whole sequence, future included, and would have handed the baseline information the learned system never has.

VIX-tercile gate: The simpler statistical gate classes each day by where the previous day's VIX level sits against the terciles of its own trailing year. A one-hot membership over low, mid, and high. The level itself is lagged one day and the tercile boundaries are lagged one day beyond the window that formed them, so a value is never compared against a window that contains it.

Gate Analysis: To understand the underlying significant factors of the gates we need a single number, comparable across schemes, saying how much of a given market variable a labelling accounts for. Each weekly decision day t carries a regime label $k_t \in \{1, \dots, K\}$ from one scheme and a value v_t of one market variable, like trailing realised volatility or the five-day return. We can treat the K regimes as groups, the total variance v will split into a part explained by the labeling and a part left to the regime, and η^2 is key quantifying metric,

$$\eta^2 \;=\; \frac{\mathrm{SS}_{\text{between}}}{\mathrm{SS}_{\text{total}}} \;=\; \frac{\sum_{k=1}^{K} n_k \big(\bar{v}_k - \bar{v}\big)^2}{\sum_{t=1}^{T} \big(v_t - \bar{v}\big)^2}, \tag{3.13}$$

where n_k is the number of weeks in regime k, $\bar{v}_k$ the mean of v over those weeks, and $\bar{v}$ the mean over all T weeks. It is bounded in $[0, 1]$: at 0 the regimes are uninformative about v, since every regime has the same mean. At 1 the labelling separates v perfectly. Reading η^2 down a column of variables therefore says what a scheme is tracking, and reading across schemes says whether two of them track the same thing.

3.6 Evaluation protocol

Walk-forward folds: Models are evaluated on a rolling walk-forward, illustrated in Figure 3.3. Each fold trains on all data up to a point, uses the following calendar year for validation and early stopping, and reports on the year after that. A short portion is removed from the tail of each training window so that nothing with memory crosses the train-validation boundary. Adjacent folds share a year, since the test year of one fold is the validation year of the next. I quantify that overlap rather than hide it. Also each evaluated year starts from cash for both models and baselines.

Aggregation: Per-fold Sharpe ratios are not simply averaged. The Sharpe ratio is a non-linear statistic, and averaging it across years will account for wealth each January and weights a calm year equally with a crisis year. Instead each seed's fold segments are stitched into one continuous daily stream and scored once, which is exactly how a baseline running continuously would be scored. Per-year figures are reported beside the headline as a consistency breakdown, never as the headline itself. Reported Sharpe ratios annualise with $\sqrt{252}$ for comparability with the literature.

The holdout: A final block of data is held out and firewalled in code. It is removed from every artifact as it is loaded, so those dates are absent from memory in every run rather than merely unevaluated, and a guard raises an error if any evaluation window crosses the boundary. The holdout is touched exactly once, by a dedicated run, after the configuration has been frozen, and its result is reported against the trial count as it stands at that moment. It is expected to carry little statistical weight, being a single short path, but high credibility, since nothing about the configuration can have been chosen with knowledge of it.

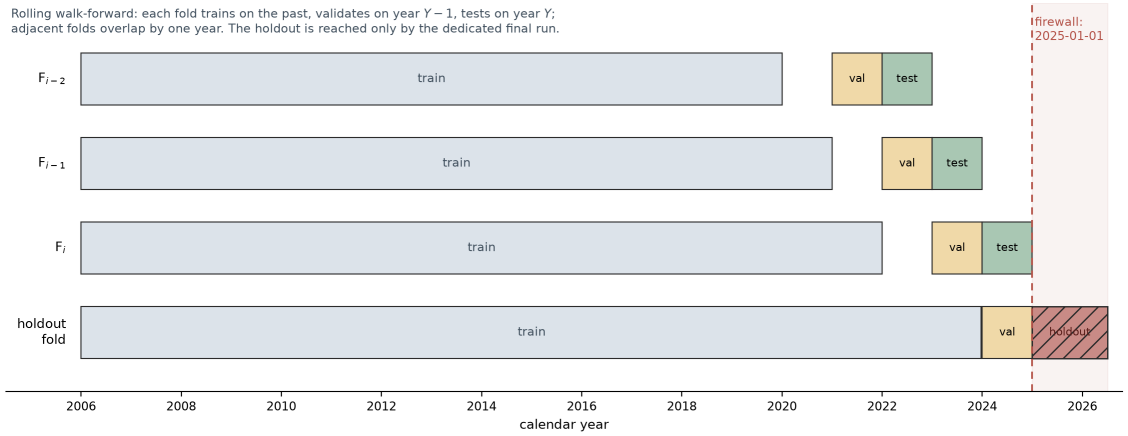


Figure 3.3: The rolling walk-forward. Each fold trains on the past, validates on year $Y-1$ and tests on year Y; adjacent folds overlap by one year. The final period is firewalled as a holdout, it is removed from every artifact at load time and reached only once, by a dedicated final run.

CHAPTER 4

Data and Experimental Setup

This chapter describes which data I take and how I process it, and how everything is set up. This includes the two chosen universes, the market-level signals I am considering, the hyperparameters of the reservoir, initialization of training heads, and the strict reproducibility discipline that every experiment in Chapter 5 obeys.

4.1 Universes and calendars

For the universe, I needed something that has relatively fixed inputs over time because the reservoir is static and can only have a fixed number of inputs. If we chose, say, the Dow 30, then since it keeps changing, taking the current top values would introduce confirmation bias. Thus, I choose two universes and further select them so they are deliberately different in character. The first, `etf10`, is a cross-asset universe of ten exchange-traded funds spanning equities, gold, commodities, treasuries, etc. It has obvious safe assets so a model can express a risk-off view by rotating into, say, bonds or gold. The second, `sector11`, is the eleven S&P 500 sector funds, an all-equity universe with no obvious defensive asset. Then I added cash to both for realism and as a risk-free option. Table 4.1 lists both the universes except the cash column.

The two universes are kept entirely separate; they are never used together in any analysis. Additionally they run on different calendar since `etf10` stated in February 2006 while `sector11` started from January 1999, and they are different kinds of testbeds. This gives us richer context than if we were to club them together, which would also lead to the same fixed-reservoir input issue discussed above. To build the trading calendar, I take the union of days on which at least one asset is traded; this creates no dependency and thus no point of failure, except for the API that provides the trading days.

Two of the sector funds, XLRE and XLC, did not exist at the start of the sample, they enter in October 2015 and June 2018 respectively. This causes the fixed input issue. Thus I only give it the assets that are present at the inception to the reservoir but on readout I do that to the full final asset values, but this can lead to buying an asset that does not even exist yet, to resolve this I introduce the availability matrix. The availability matrix $A \in \{0,1\}^{T\times N}$ records which assets are tradeable on each day. For asset i, let τ_i and ω_i be its first and last observed trading days. Then

$$A_{t,i} = \begin{cases} 1 & \text{if } \tau_i \le t \le \omega_i, \\ 0 & \text{otherwise,} \end{cases} \tag{4.1}$$

so an asset is available for its whole active life; an interior missing print does not remove it, because the asset still existed that day. The matrix is turned into an additive logit mask,

$$m_{t,i} = \big(1 - A_{t,i}\big)\,(-10^9), \tag{4.2}$$

`etf10` (cross-asset, from 2006)

Ticker	Exposure
SPY	US large-cap equity
IWM	US small-cap equity
VGK	European equity
EEM	Emerging-market equity
TLT	Long Treasuries
IEF	Intermediate Treasuries
LQD	Investment-grade credit
DBC	Commodities
GLD	Gold
VNQ	US real estate

`sector11` (all-equity, from 1999)

Ticker	Sector
XLE	Energy
XLF	Financials
XLK	Technology
XLV	Health Care
XLY	Consumer Discretionary
XLP	Consumer Staples
XLI	Industrials
XLB	Materials
XLU	Utilities
XLRE	Real Estate (2015)
XLC	Communication Services (2018)

Table 4.1: The two universes. Every universe also carries a CASH column earning the risk-free rate. Two sector funds enter late and are handled by the availability matrix.

Signal	Source	Lag	What it measures
VIX	FRED `VIXCLS`	1 day	Implied (expected) volatility
News sentiment	SF Fed daily index	≥ 5 d	Tone of economic news
Term slope	FRED `DGS10` − `DGS3MO`	1 day	Yield-curve slope
Short rate	FRED `DGS3MO`	1 day	3-month rate (policy stance)
Credit spread	FRED `BAA10Y`	1 day	Baa corporate credit premium

Table 4.2: The five market-level signals, their sources, and the publication lag applied before use.

which is 0 where available and -10^9 where not. A masked expert then outputs $w_t = \mathrm{softmax}(z_t + m_t)$, and $\exp(z_{t,i} - 10^9)$ underflows to exactly 0, so an unavailable asset receives precisely zero weight

Figure 4.1 shows the resulting coverage. The final eighteen months, 1 January 2025 to 30 June 2026, are held out: they are removed from every artifact as it is loaded, and a guard raises an error if any evaluation window crosses that boundary, so those dates are absent from memory in all but the single holdout run.

4.2 Market-level signals

The gating network and experts may optionally concatenate a small vector of *market-level* signals to the reservoir states as input. For the gating network, since these are just additional inputs, they may or may not be used, rather than defining a regime. There are five, summarised in Table 4.2. Four are drawn from FRED and one, the sentiment index, from the Federal Reserve Bank of San Francisco.

The raw levels of these values drift enormously over the sample, thus they all are passed through the same fixed, causal pipeline. They are aligned to the trading calendar, *shifted* by its publication lag, and only then standardised by a trailing 252-day z-score. It's important to note that the order matters, if we standardise and then shift that can cause look-ahead leakage. The rolling statistics would be computed from values still to be published, so a z-score at day t could depend on the future. Thus the trailing z-score keeps all five signals on a common, regime relevant scale. Figure 4.2 shows the standardised series, we can see that volatility and credit spread spike together through the 2008 and 2020 crises, similarly short rate and news sentiment show similar correlation.

4.3 Reservoir and model hyperparameters

Table 4.3 lists the hyperparameters, their default values I used, and the range over which I searched. As discussed in 3, only the gate and expert head are trained. For the reservoir, I search only four

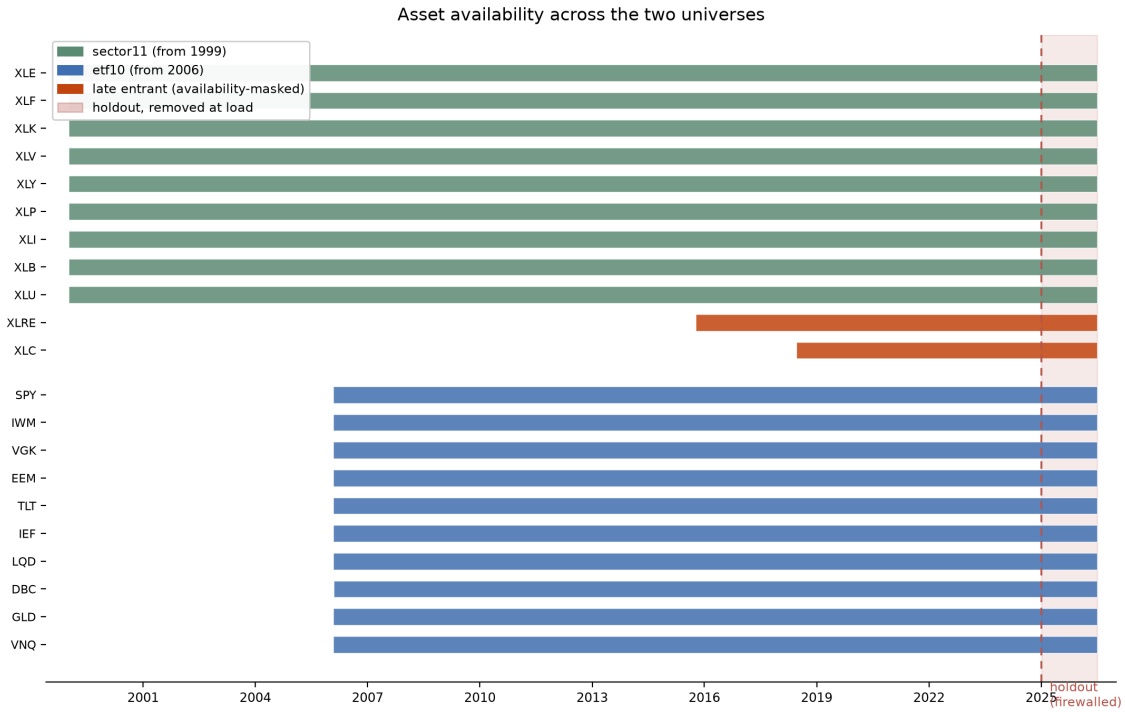


Figure 4.1: Asset availability across the two universes. `sector11` begins in 1999 and `etf10` in 2006; XLRE and XLC enter late and are availability-masked until their inception. The shaded region on the right is the firewalled holdout, removed from every artifact at load time.

hyperparameters, but as we will see in 5, since no searched configuration transfer to new data, the best choice are the defaults since they carries no selection bias.

4.4 Reproducibility

Determinism and seeds. The code is set up so a single call seeds Python, NumPy, and PyTorch, so "seed3" produces the same experiment every time. A single call seeds Python, NumPy, and PyTorch and switches on deterministic algorithms, so "seed 3" leads to the same experiment every time. Wherever logical, I report the final results as the mean and standard deviation over the 5 seeds $\{0, \ldots, 4\}$. The Simple Cycle Reservoir is deterministic, as mentioned in 3, so it does not introduce variance across runs, and its states are cached once per configuration.

The variant registry. I made a registry where the only allowed function is append. Whenever a run starts (i.e., with a different fold, split, case, etc.), it gets added to this registry. Each row records what was run, the commit, a hash of the exact configuration, and what came out (e.g., performance metrics). The configuration hash is the SHA-256 of the canonical JSON of the config, so two runs share a hash if and only if they are the same experiment, and any change to a hyperparameter produces a new hash. The entire project holds **over 3000 values** in that registry.

Guarding against selection. Because the study searches over many configurations, I treat any figure obtained by picking the best as provisional until I confirm it on data outside the one used for selection. Two mechanisms provide that confirmation: first, I re-score every searched winner on folds excluded from all selection. If it does not transfer, I do not report it as a finding. Second, the firewalled holdout of Section 4.1 is touched exactly once, after the configuration has been frozen, and its result is reported. Together, these ensure that a result which merely reflects the amount of searching I did is caught before it reaches the conclusions.

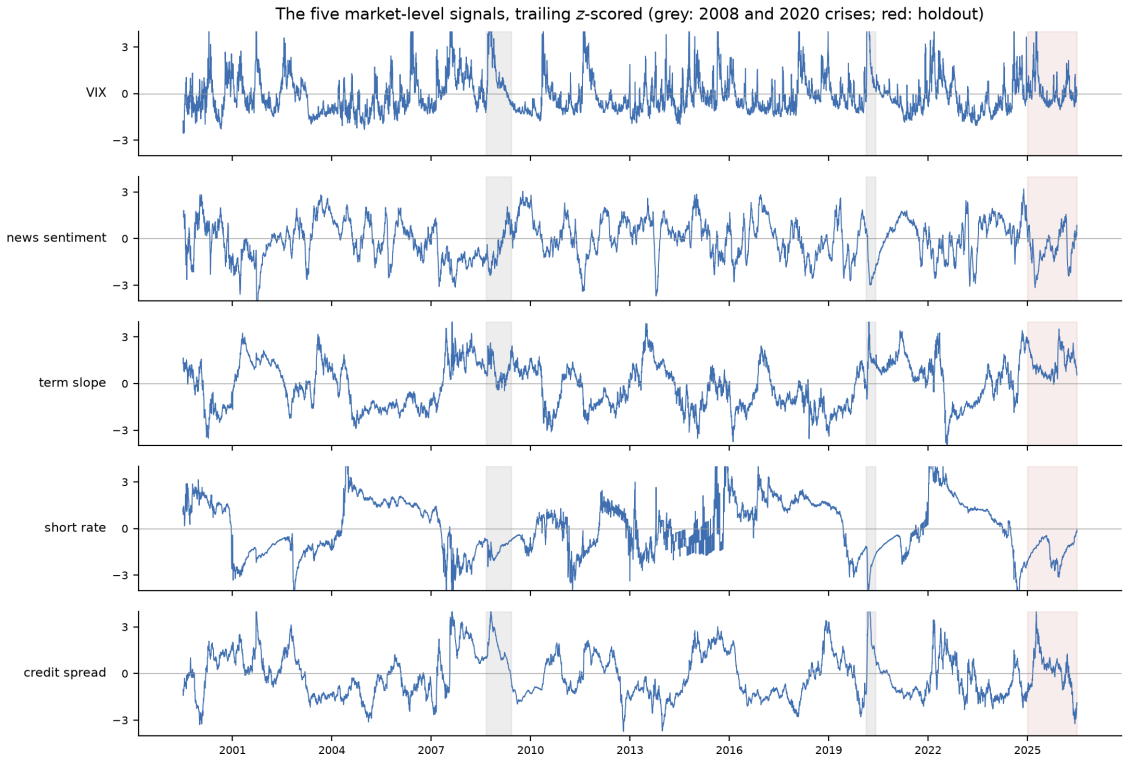


Figure 4.2: The five market-level signals after publication lagging and trailing z-scoring. Grey bands mark the 2008 and 2020 crises; the red band is the firewalled holdout. Standardisation places every signal on a comparable, regime-relative scale.

Component	Hyperparameter	Value (searched range)
Reservoir	size N_r	300 (100–1200)
	spectral radius ρ	0.90 (0.5–0.99)
	leak rate α	0.30 (0.05–1.0)
	input std target	0.70 (0.3–1.5)
(fixed)	bias scale, washout	0.0, 250 days
	calibration fraction	0.60
Model	experts K	1 (`sector11`) / 3 (`etf10`) (1–8)
	gate temperature τ_g	1.0 (0.7–1.4)
Loss	λ_{cvar}, λ_{turn}, λ_{div}	0
	λ_{bal} (only at $K > 1$)	0.01
Training	learning rate	3×10^{-3}
	max epochs	500
	max patience	25 (early stopping)
	gate noise σ_0	$0.3 \to 0$
	seeds	$\{0, \ldots, 4\}$

Table 4.3: Hyperparameters. Only the gate and experts are trained; the reservoir is frozen. The reported configuration is the default, not a searched winner.

CHAPTER 5

Results

This chapter reports the findings of the various experiments, in the order of the research questions. Throughout, a reported Sharpe ratio means the annualised excess Sharpe of the concatenated test stream, calculated from Eq. 3.2. Learned arms are the mean over the 5 seeds $\{0, \ldots, 4\}$ with the standard deviation across seeds. Baselines are deterministic and have no seed spread (except LSTM). Differences between arms are tested with the paired stationary bootstrap [14] and the Ledoit-Wolf HAC Sharpe-difference test.

5.1 What the regimes track

RQ1 asks whether the regimes discovered by our method(rung 2) differ from statistically defined ones. To start, I choose $K = 3$, which gives three gates for the three cases; I first plot them to get a visual understanding, particularly of how much they switch, as shown in Fig. 5.1. Then I compute the pairwise correlations among the three. For this, I did two things: first, I found the percentage of overlapping area between pairs, and second, I computed the AMI.

The partitions are effectively orthogonal. The key finding is from Fig. 5.2 on `sector11` the learned gate agrees with the HMM at AMI +0.004 and with the VIX terciles at +0.008; on `etf10` the values are −0.002 and +0.020. These are very small values and likely just by chance. However between the HMM and the VIX gate agree with each other at +0.093 on `sector11`, an order of magnitude above either of their agreements with the learned gate. On `etf10` the control pairing is weaker (+0.017), so the sub-claim that the two statistical schemes cluster together holds only on `sector11`.

This can also be observed visually from Fig. 5.2 , if we see the chart, if two methods were correlated then that means knowing which regime lead in one tells significant information of which regime leads in other. So we say take a column then one of this row should have significantly larger values then the others. This is only seen in the third chart, thus the two statistical gates have more in common with each other then with our gate.

Each scheme measures a different quantity. Using the method described in 3.5, I estimate the underlying significance value for each of the three ways I generate the gates. From this, we see that the statistical gates are risk detectors: the HMM labeling explains 44.3% of trailing realized volatility on `sector11` (25.2% on `etf10`), and the VIX gate explains 65.4% of its own index. For our gate, the best explanation appears to be that it is a direction detector, since its leading variable is the trailing five-day return, at 12.5% on `sector11` and 24.8% on `etf10`. At the same time, it explains under 1% of realized volatility.

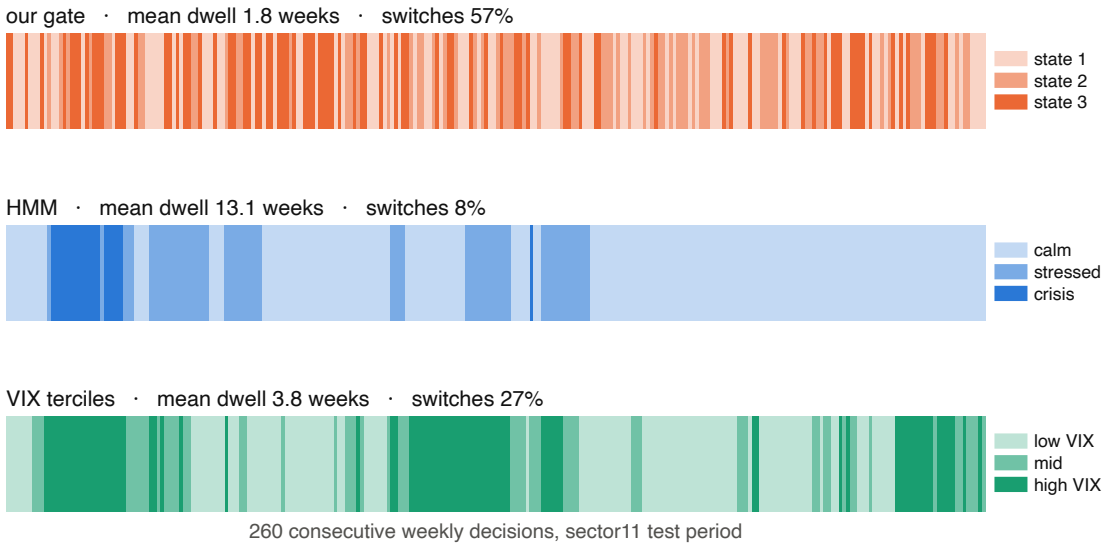


Figure 5.1: Weekly regime labels over five consecutive years of the `sector11` test period. Over the full test period the mean dwell times are 1.8, 12.9 and 3.7 weeks for the learned gate, HMM and VIX terciles; within the five-year window shown they are 1.8, 13.1 and 3.8.

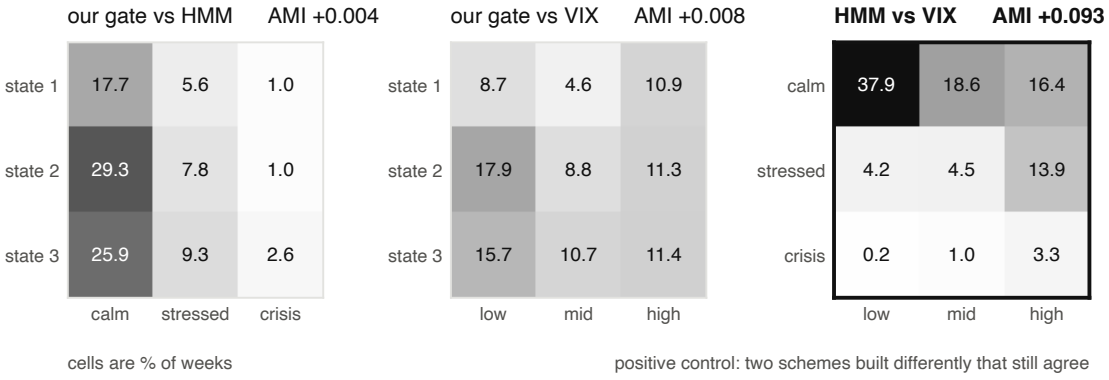


Figure 5.2: Co-occurrence of weekly regime labels on `sector11`, as a percentage of weeks, with the chance-corrected agreement (AMI) above each panel.

Equal information: adding market signals as well. The comparison so far is open to one objection: the statistical gates read market observables, while the learned gate reads only the reservoir state, so perhaps it tracks direction because it never saw a risk variable. The light-orange bars in Figure 5.3 close this. The same mixture is re-trained with the five market-level signals, VIX included, concatenated to the gate's input (the routing of Section 3.3). We see the gate using the new labelling because its AMI with preceding state only gate is 0.165–0.180 rather than 1. The metric with the largest change is the short rate whose η^2 rises by around 8%. But it does not reorganise with respect to risk. Recent direction remains its leading variable (18.4% on both universes), realised volatility stays at the floor (1.6% and 1.1%, still roughly twenty-five times below the HMM). Even agreement with the statistical gates stays at chance (AMI ≤ 0.022).

5.2 The walk-forward ladder

RQ2 asks whether the carving pays. Table 5.1 and Figure 5.4 report every arm on the full walk-forward test span: eighteen folds on `sector11` (test years 2007-2024, 4530 trading days) and ten

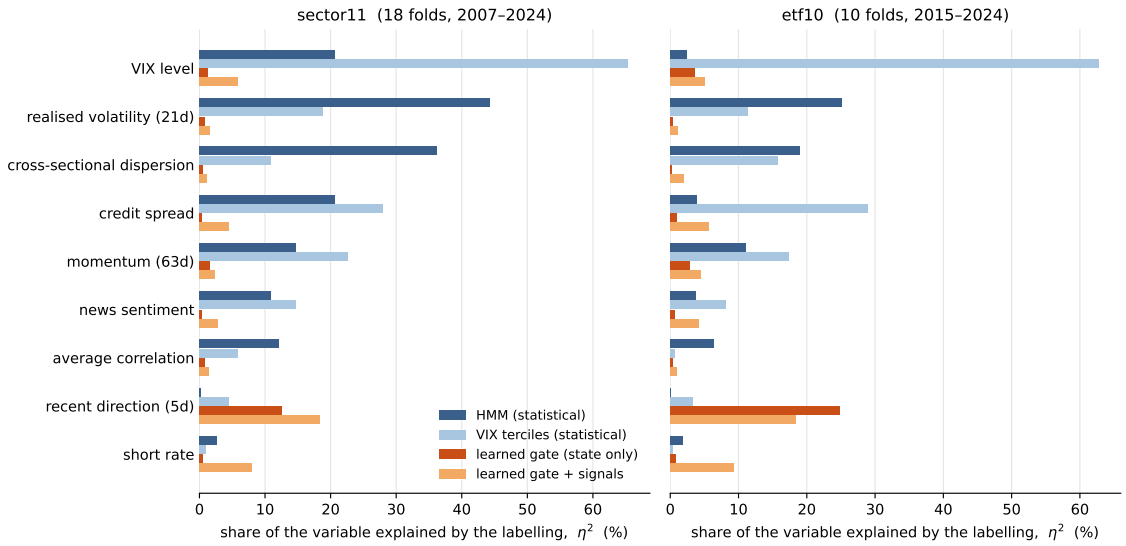


Figure 5.3: The share of each market variable explained by each labelling. The light orange bars are the same gate re-trained with all five market-level signals of Section 4.2 appended to its input: direction remains its leading variable and realised volatility stays at the floor. The figure labels abbreviate the five-signal vector as "signals".

on `etf10` (2015-2024, 2516 days).

No learned arm beats the best baseline on either universe. On `sector11` the best learned arm is the single head with the signals appended (0.507 ± 0.027), ties the inverse-volatility baseline (0.511) while trading nearly six times as much. On `etf10` it is not close: SPY scores 0.688 and the best learned arm 0.318. The two statistical gates do no better (0.418 and 0.408 on `sector11`; 0.312 and 0.232 on `etf10`), so the null is not specific to how the regimes are formed: conditioning on regimes, however defined, did not pay here i.e. none of the learned differences is significant. Each arm is compared with its neighbor on the ladder, the net difference in max and min of ours is only about 0.1.

On account of money gained, the seed-0 stream of the best learned arm turns £10,000 into £45,998 on `sector11` against £54,551 for buy-and-hold SPY, and into £15,545 on `etf10` against £34,059 (Figure 5.5). From the figure two interesting things are seen. First, on `sector11` the learned arm is ahead of SPY for most of a decade and the lead changes only after 2021. I do not read this as a regime the model exploited and then lost rather for such wide intervals one path crossing another for years at a time is what a tie looks like. Second, in 2008 the learned arm rides the crash down with the market, losing roughly half its value; its drawdown (−49%) is close to SPY's (−55%). Meaning the gating detects nothing that would have removed it from harm's way, which is consistent with Section 5.1: a direction detector reacts to a crash once prices have already fallen.

Additional context to the results of `etf10` On `etf10` every diversified allocation lost to SPY over 2015-2024: 1/N scores 0.374, inverse-volatility 0.278 and Ledoit–Wolf mean-variance 0.058, because bonds fell as rates rose and non-US equity and commodities lagged US large-caps for the whole decade. The learned arms sit beside 1/N, which is what they learned to be. The honest reading is not that the model is uniquely broken on `etf10` but that diversification itself lost to the index in that window, and the model paid the same price plus turnover.

5.3 Capacity, selection and transfer

RQ3 asks whether more capacity or more information helps: more experts, a trained encoder(LSTM) in place of the frozen reservoir, extra signals. The signal part was answered negatively above. This

	`sector11` (2007–24)			`etf10` (2015–24)		
arm	Sharpe	turn	maxDD	Sharpe	turn	maxDD
1/N	0.485	1.3	−53%	0.374	1.3	−23%
buy-and-hold SPY	0.510	1.0	−55%	**0.688**	1.0	−34%
inverse-volatility	0.511	1.7	−50%	0.278	1.8	−21%
Ledoit–Wolf MV	0.479	4.5	−35%	0.058	4.4	−38%
ours, K=1†	0.414 ± 0.019	12.5	−51%	0.296 ± 0.028	8.4	−23%
ours, K=3‡	0.461 ± 0.030	10.3	−49%	0.237 ± 0.025	8.1	−23%
3a signals → gate	0.488 ± 0.045	9.5	−50%	0.280 ± 0.057	7.5	−23%
3b signals → experts	0.469 ± 0.063	8.8	−47%	0.272 ± 0.036	9.4	−23%
3c signals → both	0.500 ± 0.043	9.6	−44%	0.264 ± 0.017	7.5	−23%
ours, K=1 + signals	0.507 ± 0.027	9.8	−49%	0.318 ± 0.034	8.1	−22%
HMM gate	0.418 ± 0.015	11.7	−47%	0.312 ± 0.020	10.6	−20%
VIX gate	0.408 ± 0.036	12.4	−52%	0.232 ± 0.022	11.4	−29%
LSTM (trained encoder)	0.378 ± 0.017	4.0	−50%	0.299 ± 0.027	4.4	−19%
shown for reference only; values with best found K						
best capacity§ (K=2 / K=5)	**0.513** ± 0.025	—	—	0.408 ± 0.106	—	—

Table 5.1: Concatenated test Sharpe (mean ± standard deviation over five head seeds), annualised one-way turnover and maximum drawdown. The best value in each Sharpe column is in bold. † frozen configuration on `sector11`; ‡ frozen configuration on `etf10` (Table 4.3). "Signals" is the five-signal vector of Section 4.2. (Section 5.3).

search	universe	encoder	selection	held out	untuned ref.
grid, $N_r \times \rho$	`sector11`	reservoir	0.603 ± 0.02	0.526 ± 0.09	0.534 ± 0.02
joint, $K \times \rho$	`sector11`	reservoir	0.582	0.428	0.509
random, 50 draws	`etf10`	reservoir	0.438 ± 0.07	0.929 ± 0.15	0.811 ± 0.05
joint, $K \times \rho$	`etf10`	reservoir	0.464	0.783	0.809
staged, 16+ cells	`sector11`	LSTM	1.011	0.738 ± 0.22	0.554 ± 0.04
staged, 16+ cells	`etf10`	LSTM	1.068	0.473 ± 0.04	0.605 ± 0.06

Table 5.2: Every configuration selected by a search, re-scored on folds excluded from all selection (`sector11` F13–F18, `etf10` F06–F10). "Selection" is the winner's score on the folds it was selected on; "held out" its score on the excluded folds; "untuned ref." the untouched reference on the same held-out folds (the Appendix-B reservoir defaults, or for the LSTM the fixed ladder configuration). Values without a spread are single re-scored runs.

section reports impact of the other two.

Impact of more experts. Figure 5.6 gives each $K \in \{1, \ldots, 8\}$ its own small reservoir hyperparameter search and scores each winner on the full test span. On `sector11`, $K = 1$ is the worst capacity on test (0.403) and $K = 2$ the best (0.505); $K = 2$, 5, 7 and 8 all resolve above $K = 1$ by more than two seed spreads. The searched reservoir width also grows with K (from 300 to 600 units).

Selection on validation points the wrong way. The right panel of Figure 5.6 is the sharpest result in this chapter. Across the eight capacities, validation and test Sharpe have negative correlation at $r = -0.477$: $K = 4$ has the best validation score of all eight and lands mid-table on test, while $K = 2$ has nearly the worst validation and the best test. On `etf10` the correlation is positive ($r = +0.678$), so the anti-correlation is a `sector11` finding and not a law.

No searched winner transferred. Table 5.2 re-scores every searched winner on folds that took no part in any selection. On `sector11` both reservoir winners come back at or below the untuned defaults, one of them badly (0.582 on its selection folds, 0.428 held out, against the defaults' 0.509). On `etf10` the random-search winner comes back 0.118 ahead of the defaults, inside one seed spread (0.150), which is no evidence of transfer rather than evidence of failure; the joint winner comes back slightly behind. The two LSTM champions are the largest reversals: validation scores of

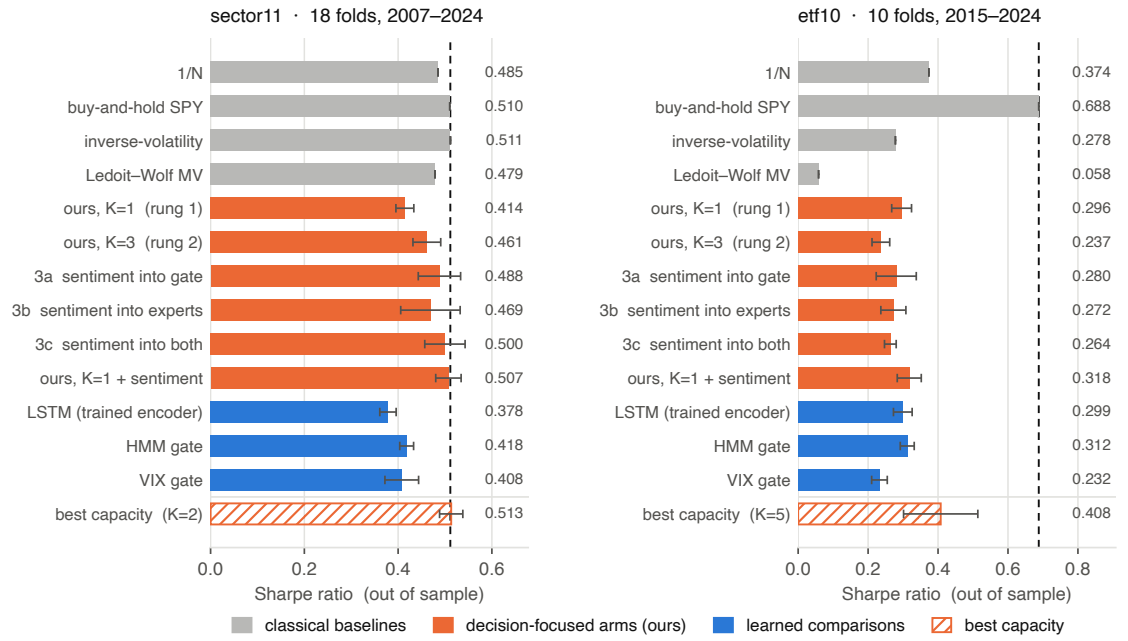


Figure 5.4: The ladder of Table 5.1. Error bars are seed dispersion, not significance intervals; the baselines are deterministic and show none. The dashed line marks the best baseline in each universe. The "rung" numbers in the labels index the build order of the study, and "sentiment" in the arm labels abbreviates the five-signal vector.

1.011 and 1.068 collapse to 0.738 and 0.473 held out, the latter below even the untuned ladder configuration.

A dense grid finds nothing to fix. A final, denser search was performed with 1,485 configurations in a local grid around each universe's best values (810 on `sector11`, 675 on `etf10`), validated on the selection folds, with the top twelve re-scored on the full test span. Neither winner survived the move. On `sector11` the best cell on validation scored 0.678 there and 0.522 on test, against the 0.505 the configuration already in hand reached. The highest test score among the twelve was 0.532, but that is the largest of twelve noisy numbers, and a single number here moves by 0.02 to 0.06 on the choice of seed alone, so it is not meaningfully better. On `etf10` the gap is wider: the validation winner scored 0.659 and then tested at 0.250, well below the 0.411 already in hand, which remained the best test score found anywhere in the grid.

So searching harder only lifted the validation ceiling, from 0.559 to 0.678. The test scores did not follow it up. Figure 5.7 shows why by showcasing that for one slice of the grid the settings that do well on validation and the settings that do well on test are not the same settings.

The trained encoder. Till now I gave the LSTM one hand-picked configuration while the reservoir side had run hundreds of search trials, so the comparison was not yet fair. A matched search fixes this. Here the size of hidden state and lookback matters. When we increase hidden states from 16 → 128 it raises validation Sharpe by 0.456 on `sector11` and 0.347 on `etf10`. The impact of lookback is much lower, increasing it from 20 → 250 moves it by at most 0.067. So the 60-day window was never the handicap, which the reservoir's own memory capacity corroborates. But that capacity bought validation score that did not last. Both champions fall from 1.011 and 1.068 to 0.738 and 0.473 on the folds held out of the search (Table 5.2), and on the ladder's own metric they reach only 0.237 and 0.254 where the pre-specified configuration reaches 0.378 and 0.299.

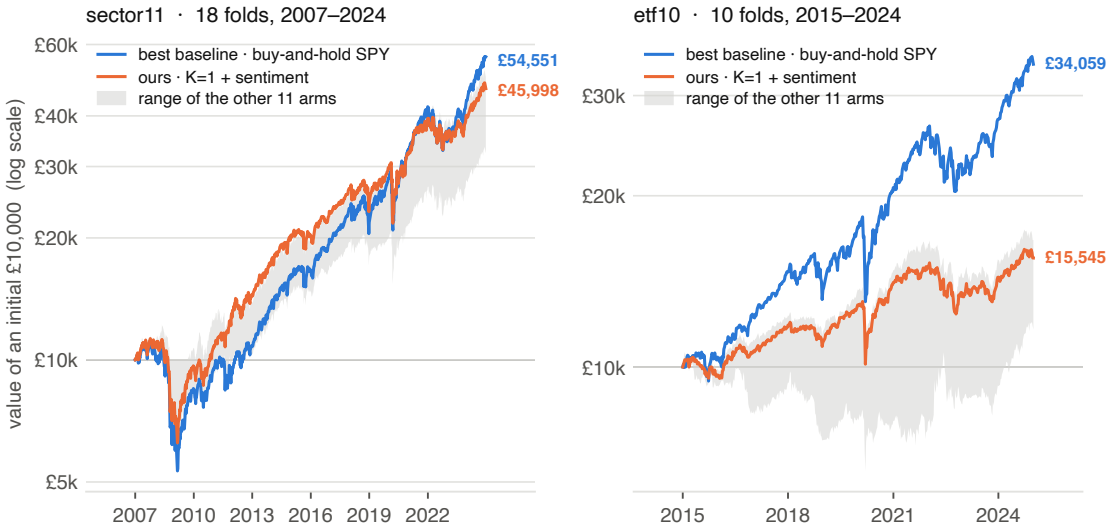


Figure 5.5: Growth of an initial £10,000, using the seed-0 stream of the best learned arm against the best baseline; the grey band is the min–max envelope of the other eleven arms.

penalty	$\lambda = 0$	small	large
λ_{cvar} (1.0 / 5.0)	**0.526**	0.505	0.449
λ_{turn} (0.5 / 2.0)	**0.526**	0.403	0.412
λ_{div} (0.01 / 0.05)	**0.547**	0.526	0.443

Table 5.3: Validation-era sweeps of the three loss penalties on `sector11` (concatenated metric; the two sweeps in the upper rows share a base configuration, the third ran at a slightly different one). Every penalty is best at zero. The turnover penalty cuts turnover from 9.2 to 1.8 and costs 0.12 of Sharpe; weight decay and a drawdown penalty, swept separately, also select to zero.

Effect of regularisation. If the train validation gap was due to overfitting, a penalty function can be used to reduce it and improve performance. Five penalties were implemented and swept: CVaR, turnover, expert diversity, weight decay and drawdown. All five select to zero (Table 5.3). Weight decay is the penalty on larger weight, thus this should have best impact on closing the train validation gap from 0.84 to −0.11, yet final validation still gets worse. Consistently, raising K adds parameters and widens the gap (from −0.006 at $K = 1$ to +0.047 at $K = 6$) while train Sharpe does not improve at all (0.482, 0.476, 0.462 at $K = 1, 3, 6$) meaning the extra capacity buys variance, not fit. Chapter 6 argues that this pattern, no penalty computed on the training window helps, is what non-stationarity looks like, as opposed to overfitting.

5.4 The holdout

The final eighteen months, 2 January 2025 to 30 June 2026 (373 trading days, 78 weekly decisions), were firewalled from the start of the project (Section 4.1) and evaluated once, by a dedicated script. Since I found no transference and optimal hyperparameters I use the initial best found values as frozen defaults. Training data end in December 2023, and 2024 served as validation. Alongside the frozen primary ($K = 1$ on `sector11`, $K = 3$ on `etf10`, all defaults) I ran one searched secondary, the best-capacity winner of Section 5.3, to measure what selecting the searched configuration would do, and the four baselines.

On top we see 1/N on both universes, and our frozen configuration lands closely behind with 0.811 against 0.812 on `sector11` and 1.295 against 1.340 on `etf10`. Thus the learned system performs like a diversified baseline, no better and no worse. For a study whose main performance result is a null, this is the preferable outcome, the holdout confirms what we have discussed till now instead of contradicting it.

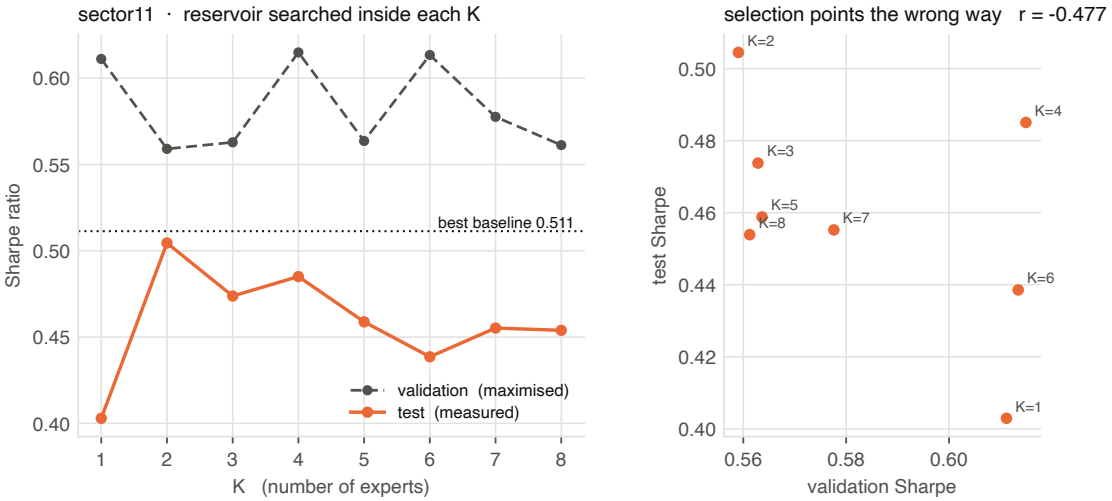


Figure 5.6: Left: validation and test Sharpe against the number of experts K on `sector11`, with the reservoir searched inside each K and the winner scored on the full walk-forward test. Right: the same eight points as a scatter, validation and test anti-correlate ($r = -0.477$)(Section 5.3).

`sector11`	Sharpe	maxDD	turnover
1/N	**0.812**	−15.0%	1.1
frozen K=1 (primary)	0.811 ± 0.033	−13.7%	3.6
searched K=2 (secondary)	0.809 ± 0.022	−13.7%	2.9
buy-and-hold SPY	0.786	−18.8%	0.7
Ledoit–Wolf MV	0.777	−14.5%	5.7
inverse-volatility	0.710	−14.4%	1.5
`etf10`			
1/N	**1.340**	−8.9%	1.0
Ledoit–Wolf MV	1.313	−19.5%	2.0
frozen K=3 (primary)	1.295 ± 0.023	−8.3%	3.0
inverse-volatility	1.136	−6.7%	1.5
searched K=5 (secondary)	0.930 ± 0.583	−6.8%	14.6
buy-and-hold SPY	0.786	−18.8%	0.7

Table 5.4: The single holdout run, 2025-01-02 to 2026-06-30. SPY scores identically on the two universes because it is the same asset on the same days, thus this serves like an internal consistency check of the pipeline.

On `sector11` the searched configuration does no better than the frozen one (0.809 against 0.811); on `etf10` it is worse and wildly unstable (0.930 ± 0.583 across seeds, at a turnover of 14.6). Had I put a searched winner forward as the primary, the holdout would have returned a worse and less reliable number. One small observation runs in the model's favour: on both universes the frozen arm's maximum drawdown is shallower than that of every baseline that matches or beats its Sharpe.

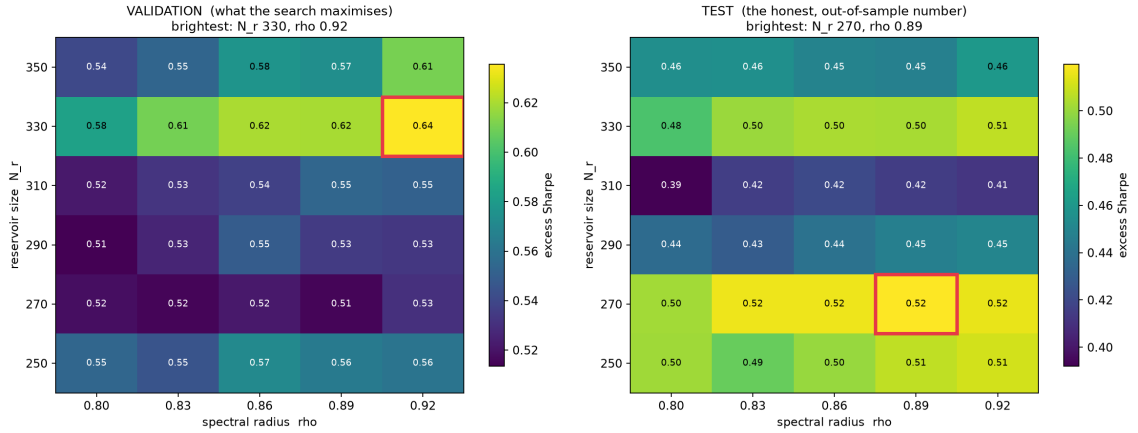


Figure 5.7: The $N_r \times \rho$ slice of the fine grid at default parameters, `sector11`, $K = 2$: validation Sharpe (left) and test Sharpe (right) on the same cells, each panel's best cell boxed. The maximum do not coincide, and the validation best row ($N_r = 330$) is mediocre on test while the validation worst row ($N_r = 270$) is the test-hottest.

CHAPTER 6

Discussion and Limitations

6.1 Why the carving differs

The η^2 analysis of Section 5.1 says ours vs statistical gate are measuring different quantities. The statistical gates partition the sample on risk, our gate biggest found focus is recent direction, and each explains almost nothing of the other's variable. Once that is seen, the rest of the RQ1 evidence follows. Two labellings driven by nearly unrelated variables must agree at chance, which is the AMI ≈ 0 of Figure 5.2. And the dwell times follow the variable each gate watches: volatility persists for months, so the HMM holds a state for 12.9 weeks, while five-day direction flips constantly, so our gate holds one only for 1.8 (Figure 5.1).

In simple terms our gate's three states have mean five-day returns of $+0.34\sigma$, $+0.02\sigma$ and -0.55σ on `sector11`, with every other variable inside 0.15σ, so they can be read as "just rose", "drifted" and "just fell" (note that this is not the full picture since even 5 day direction's value in Fig 5.3 is not that large) . The HMM's states read more like calm, stressed and crisis, with mean realised volatility -0.40σ, $+0.96\sigma$ and $+1.62\sigma$ and mean direction below 0.1σ in every state: it cannot tell you which way the market went, and our gate cannot tell you how rough the ride was.

There is a structural reason our gate could not have produced the HMM's partition. With zero bias and $x_0 = 0$ (Table 4.3), the reservoir state is an *odd* functional of the driving returns i.e. flip the sign of every input and every state flips sign,

$$x_t(-u) \;=\; (1-\alpha)\,x_{t-1}(-u) + \alpha\tanh\big(W x_{t-1}(-u) - v\,W_{\text{in}}u_t\big) \;=\; -\,x_t(u), \tag{6.1}$$

because tanh is odd and every other operation in Eq. 3.5 is linear. Now direction is an odd function of returns, while any volatility measure is even, meaning it remains unchanged when every return flips sign. A volatility partition must therefore give u and $-u$ the same label. Consider the following case:

The regime label at week t is the expert with the largest $\text{argmax}_k\,(W_g x_t + b_g)_k$ (Eq. 3.8). Under the flip of Eq. 6.1 the bias is unchanged while the state term reverses, so the mirrored week is scored with $b_g - W_g x_t$. Take $K = 3$ & zero bias

$$\underbrace{(+2,\,-1,\,-1)}_{\text{Week A}} \qquad \longrightarrow \qquad \underbrace{(-2,\,+1,\,+1)}_{\text{Week B}}. \tag{6.2}$$

The two weeks have identical volatility, yet the gate labels them differently, so this labelling is not a volatility partition. Thus because of the odd like (because of the bias term, if it is 0 then its odd nature) nature of both reservoir and our gate we can't account for volatility. The HMM faced no such constraint because it was handed trailing realised volatility as an observable (Section 3.5) thus its risk partition was half-built before fitting began.

When the five market-level signals are appended to the gate's input, the even-function barrier is gone: VIX arrives as an ordinary linear feature. The gate does read the new inputs, but we see that what it actually absorbs is the short rate, while realised volatility stays at the floor and direction stays first (Figure 5.3). So the architecture could not build a risk detector from the state, and, offered risk directly, the decision objective still declined to become one.

6.2 Why it does not pay

The obtained performance null was not just one failed experiment, rather it is the same answer arriving from different directions. The universe is not the bottleneck: the natural reading of 2008 on `sector11`, that an all-equity universe simply offers no defensive action, predicts the models should do better on `etf10`, which holds Treasuries and gold, and instead they lose there by more, ending with roughly half of SPY's terminal wealth (Section 5.2). Capacity is not the bottleneck: train Sharpe is flat in the number of experts (0.482, 0.476, 0.462 at $K = 1, 3, 6$) while the train validation gap widens, so the added parameters buy variance rather than fit, and even the test selected best capacity only lands on top of the best baseline (0.513 ± 0.025 against 0.511, Table 5.1). Information is not the bottleneck: the signal added single head ties a baseline at six times the turnover, and handing the signal vector to the gate at those same best-capacity configurations moves test Sharpe by -0.023 (paired seed sd 0.026) on `sector11` and -0.207 (sd 0.140) on `etf10` .A gate that will not reorganise onto risk cannot profit from a risk signal, and the extra inputs only give it more to overfit. The encoder is not the bottleneck: the LSTM's search buys validation Sharpe of 1.011 and 1.068 that becomes 0.237 and 0.254 on test, below the small pre-specified network. And the gap is not fixable overfitting: all five penalties select to zero (Table 5.3).

Weight decay is the observation that tackles overfitting. If overfitting was the culprit then, shrinking of weights should likely close the train-validation gap *and* raise validation. Weight decay closes the gap completely, from 0.84 to -0.11, still validation gets worse. The gap is therefore likely not the model over-learning a stable target rather the target itself moves between the training years and the year that follows. Every penalty is computed on the training window, so none of them can help with a problem that only exists outside it. That is the sense in which non-stationarity is the binding constraint on this problem at this data size.

From all of this we can conclude, what the system converges to is reasonable rather than broken. With some structure that survives from one year to the next, the decision objective settles into a diversified, yet nearly static book: the learned arms sit beside 1/N on the walk-forward and tie it on the holdout (Table 5.4). And the coordinate it does condition on cannot rescue it in a crisis, because a direction detector reacts once prices have already fallen, which is why the learned arm rides 2008 down with the market (Section 5.2).

6.3 Methodological lessons

The transfer failures of Section 5.3 have a common mechanism. A validation score is not a measurement of a configuration, rather it is the maximum of many noisy numbers. Early stopping keeps the best of hundreds of epochs, and a search keeps the best of many configurations, so the reported winner is selected twice over, and the maximum of a noisier sequence is larger. That is why more capacity manufactures more validation score and keeps less of it: the LSTM, the highest-capacity model in the study, shows the largest validation numbers and the largest collapse (1.011 to 0.237 on test).

Selection then fails in a specific way, out of sample, the spread across searched configurations (0.064) is smaller than the spread between two seeds of one configuration (0.085), meaning which configuration we pick matters less than which seed we draw, so the argmax of validation is essentially a draw from noise. On `sector11` it is worse, validation and test anti-correlate across capacities ($r = -0.477$, Figure 5.6), so selecting K on validation would not merely have failed to help, it would have pointed away from the answer. The interaction found by the capacity curve compounds this, the best reservoir changes with K.

Two smaller lessons are about sample size, a search scored at a single seed froze a winner at validation 0.817 that scored 0.298 when re-run across seeds; and a three-seed spread of 0.007

became 0.088 when one point was re-run at eight seeds, moving the point itself from 0.215 to 0.276. Meaning small samples do not merely add noise to a finding, they manufacture findings.

What held up, in the end, was the base part of the protocol: pre-specified defaults, so every searched winner treated with doubt until re-scored on folds no selection had touched (Table 5.2), and one firewalled holdout. The holdout repaid it directly, the untuned defaults tied 1/N while the searched secondary came back worse and unstable (0.930 ± 0.583 at a turnover of 14.6). Had a searched winner gone forward as the primary instead, the one number in this study that no reader can argue with would have come back worse. On a problem that offers only a few hundred weekly decisions per fold, taking the untuned default and checking it on data no search has seen is not the weaker way of working; it is the only way whose final number means what it appears to mean.

Now I report the limitations of my project, none of these limitations, in my judgement, invalidates the conclusions, although several narrow their scope.

6.4 Evaluation design

The evaluation runs one walk-forward, which gives each arm a single backtest path. So the uncertainty I quote comes from resampling days and from changing seeds, not from the many other paths the same strategy might have taken. A combinatorial purged cross-validation would have produced a spread of such paths, and from that spread an estimate of how likely a chosen strategy is to be overfit. This is the main gap in our method, and Chapter 7 puts it first among the future work. Given how much of this study turned out to be about selection not transferring, a design built to measure that risk directly would have been the right tool for the job.

Folds also overlap by design, the test year of one fold is the validation year of the next (Section 3.6), so 7 of 18 `sector11` and 4 of 10 `etf10` test years had been seen during hyperparameter selection. No model ever trained on those years, but the search did look at them. Looking only at test years no search had touched, the learned arms gain between +0.02 and +0.06, while buy-and-hold SPY gains +0.21.

Annualising with $\sqrt{252}$ assumes days are independent. Our streams show a first-order autocorrelation of about -0.10, so the convention is mildly conservative here, and every test we run uses HAC standard errors, which assume nothing about independence. And a null result says we could not find an edge, not that there is none. With seed spreads of 0.02 to 0.09 and intervals straddling zero, anything smaller than about one seed spread would have gone unseen. What the evidence supports is that no edge shows up at this sample size, with the point estimates sitting at or below the baselines and the holdout agreeing.

6.5 Scope of the conclusions

Our study focuses on 2 US universes, the results are susceptible to being specific to this data set, this data size, training size, etc. I tried to make logical statements and generalise wherever possible, but the result may not be the same in a decision-focused regime, gating at, say, daily decision frequency or much larger assets, where the number of decisions per parameter is different by orders of magnitude.

The conclusions are also period-specific which is flagged in the results chapter. The `etf10` reading is dominated by a decade in which the index beat every diversified allocation. The `sector11` reading is shaped by 2008 sitting in its early folds. The holdout window had no such huge drop, every arm scored well (1/N reaches 1.34 on `etf10`), the standard error of a Sharpe ratio estimated from 373 trading days is close to one, and the run was scoped in advance to the frozen primary, one searched secondary and the four baselines, with no arm added after the results were seen.

6.6 Model choices

The comparison again similar methodology is only done with one kind of trained encoder, i.e. LSTM. It reads a fixed window of recent days(to have it comparable to our reservoir) and then forgets it, a stronger recurrent baseline would carry its memory forward across the whole sample

and train on that memory. The model search also tuned the remaining settings one at a time, this assumes that they do not interact much with network size. So "a trained encoder does not beat the frozen reservoir here" is established for windowed, memory less recurrent encoders trained in our loop, and not for more general models.

Our interpretability analysis runs at $K = 3$, because there are no regimes to examine unless there is more than one expert to route between and 3 are easy to classify and visualize. The frozen `sector11` configuration is $K = 1$, where a softmax over a single option is always 1.0 and the gate does nothing at all. The RQ1 findings therefore describe what the decision objective does when it is given regime capacity, they do not describe the frozen `sector11` arm, which has no regimes to interpret. In the same spirit, each statistical gate is limited, a single HMM fitted to two observables and a single VIX tercile rule, both held fixed rather than tuned. And η^2 only measures how much of a variable a labelling accounts for. It does not show that one causes the other. Finally the symmetry result of Chapter 6 explains why the carving comes out as direction, but it also means the architecture itself limits which regimes a *linear* gate can find, so this study cannot say what a nonlinear gate would have discovered. That question is left deliberately to Chapter 7.

CHAPTER 7

Conclusion and Future Work

I built a portfolio system that is trained on the quality of its own decisions rather than on forecast accuracy like with MSE. A differentiable ledger whose realized excess Sharpe is the loss, a frozen Simple Cycle Reservoir to map to a higher feature dimension, and a gated mixture of experts whose regimes are discovered by that loss itself. To test this fairly I build various helper functions like a virtual ledger that takes portfolio suggestions and run them through a fixed path for that given time range, accounting for transaction cost, trading days etc. All models including the baselines, our proposed models, and the LSTM run through this ledger. For reproducibility I set fixed seeds, various deterministic algorithms, and to verify an append-only run registry. Finally a holdout fire walled from the first day and only touched once.

Three central findings are noticed. First, regimes discovered by the decision objective are completely different from statistically defined ones, and we can even say why: the statistical gates track risk, ours tracks recent market direction, and the symmetry argument of Chapter 6 shows given the zero-bias linear gate and odd reservoir state used here, the architecture cannot represent a purely even, sign-invariant risk partition from the reservoir state alone. Second, the difference does not pay. No learned arm beats(as in is significantly better) the best naive baseline on either universe, no difference involving a learned arm is significant, and the holdout reproduces the same out of sample. Third, hyperparameter selection does not transfer on this problem meaning for either encoder, configurations chosen on validation data come back no better, and sometimes worse, than untuned defaults. I regard the honest null, delivered with its mechanism, as a result rather than a disappointment, and the evaluation protocol that made it trustworthy as a contribution in its own right.

7.1 Future work

First, as promised in Section 6.4: replace the single-path walk-forward with combinatorial purged cross-validation. It produces a distribution of backtest paths and, from it, an estimate of the probability that a selected strategy is overfit. Since our central methodological finding is that selection does not transfer, the natural next study measures that probability rather than inferring it from held-out checks.

The linear gate works like an odd function and cannot express a risk partition (Eq. 6.1). The open question is what a gate free of that constraint discovers. Two ways to do this is to either use a small nonlinear gate, or to use simple even features of the state such as its squares and then re-run through the same η^2 analysis. If the nonlinear gate still organises on direction, the revealed preference of the decision objective is confirmed. If it switches to risk, the architecture, not the objective, was the binding constraint.

The single reservoir reads a fixed-width vector, which is why the universe must be frozen at

inception. This was the reason why I chose these two universes to drive the encoder, and why a changing index like the Dow was out of reach (Section 4.1). A natural redesign gives each asset its own small cycle reservoir, with shared or individual hyperparameters, and lets the heads read the pooled per-asset states. An asset's reservoir then simply starts when the asset comes into the market. Assets can thus freely enter and leave, the input dimension is no longer static, and the method extends to evolving universes.

We see a large gap in the training and validation sharpe values at all K values. This is likely due to drift, the optimal distribution/regime structure , etc changes from one year to next, which is what the gap shows. One solution would be to have an objective that rewards transfer rather than fit, such as maximising the *worst* sub-period Sharpe of the training span instead of the pooled one, so that a configuration is preferred only if it survives every regime it has already seen.

Bibliography

[1] Adam N. Elmachtoub and Paul Grigas. Smart "predict, then optimize". *Management Science*, 68(1):9–26, 2022.

[2] William F. Sharpe. The sharpe ratio. *The Journal of Portfolio Management*, 21(1):49–58, 1994.

[3] James D. Hamilton. A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica*, 57(2):357–384, 1989.

[4] Ali Rodan and Peter Tiňo. Minimum complexity echo state network. *IEEE Transactions on Neural Networks*, 22(1):131–144, 2011.

[5] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.

[6] Herbert Jaeger. The "echo state" approach to analysing and training recurrent neural networks. Technical Report GMD Report 148, German National Research Center for Information Technology, 2001.

[7] Mantas Lukoševičius and Herbert Jaeger. Reservoir computing approaches to recurrent neural network training. *Computer Science Review*, 3(3):127–149, 2009.

[8] Lyudmila Grigoryeva and Juan-Pablo Ortega. Echo state networks are universal. *Neural Networks*, 108:495–508, 2018.

[9] Boyu Li, Robert Simon Fong, and Peter Tiňo. Simple cycle reservoirs are universal. *Journal of Machine Learning Research*, 25(158):1–28, 2024.

[10] Lawrence R. Rabiner. A tutorial on hidden markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989.

[11] Andrew Ang and Geert Bekaert. International asset allocation with regime shifts. *The Review of Financial Studies*, 15(4):1137–1187, 2002.

[12] Massimo Guidolin and Allan Timmermann. Asset allocation under multivariate regime switching. *Journal of Economic Dynamics and Control*, 31(11):3503–3544, 2007.

[13] Olivier Ledoit and Michael Wolf. Honey, i shrunk the sample covariance matrix. *The Journal of Portfolio Management*, 30(4):110–119, 2004.

[14] Dimitris N. Politis and Joseph P. Romano. The stationary bootstrap. *Journal of the American Statistical Association*, 89(428):1303–1313, 1994.

APPENDIX A

The second universe

The regime figures of Chapter 5 are drawn on `sector11` and the corresponding `etf10` numbers are quoted in the text. This appendix shows the `etf10` counterparts, from the same artifacts. The capacity curve is included here rather than in the main text because its three-seed spreads are too wide to rank the capacities. The error bars in Figure A.3 are the evidence for that statement.

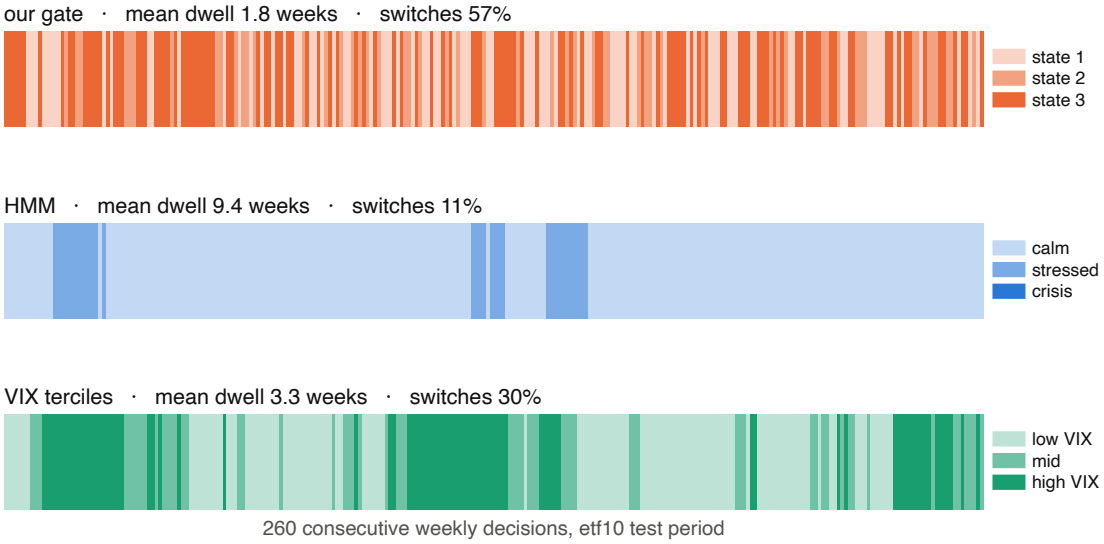


Figure A.1: Weekly regime labels over five consecutive years of the `etf10` test period. Over the full test period the mean dwell times are 1.8, 9.2 and 3.3 weeks for our gate, the HMM and the VIX terciles; the same fast-versus-slow pattern as `sector11`.

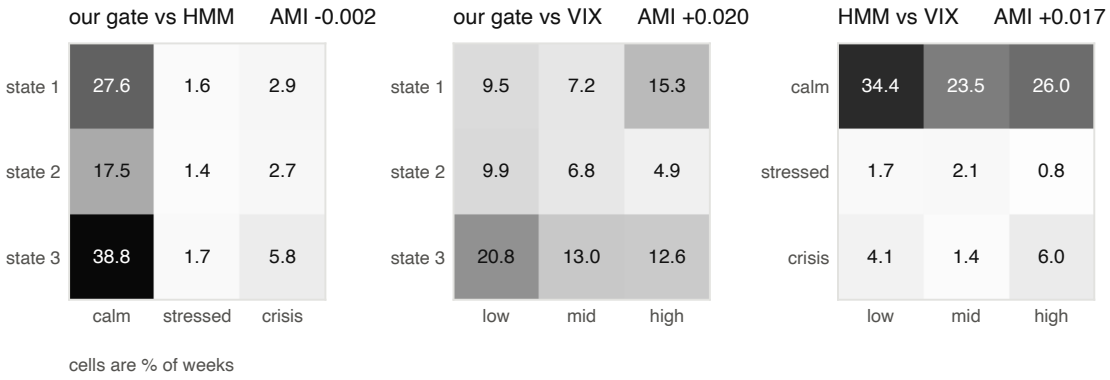


Figure A.2: Co-occurrence of weekly regime labels on `etf10`. Our gate agrees with neither statistical scheme (AMI -0.002 and $+0.020$). The control pairing is weaker here than on `sector11` ($+0.017$ against $+0.093$), which is why the main text claims the clustering of the statistical schemes on one universe only.

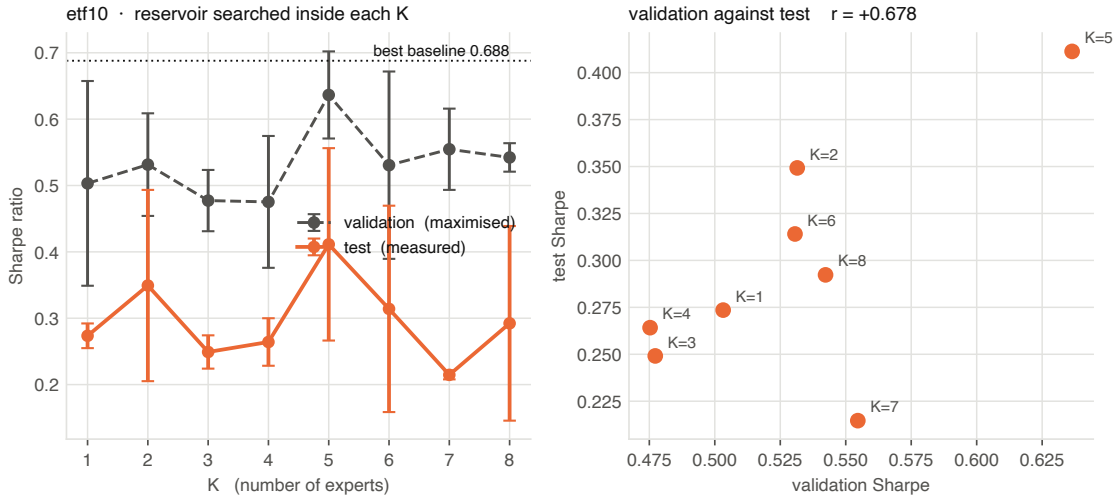


Figure A.3: The withheld `etf10` capacity curve, three seeds per point. The spreads are as large as the differences between capacities, and re-running one point at eight seeds moved it from 0.215 to 0.276 and its spread from 0.007 to 0.088, so no per-K ordering is read from this curve. Validation and test correlate positively here ($r = +0.678$), so the anti-correlation of Section 5.3 is a `sector11` finding, not a law.

APPENDIX B

Negative results and reversals

Three results were measured, but kept out of the main text. They are of lower significance and thus are noted here. First, a per-date multi-objective selection front (NSGA-II) was built in full on top of the trained blend, measured, and deleted. Restricting the blend to its largest positions costs nothing but re-optimising the positions with the front destroys performance (Table B.1). Bolting a predict-then-optimise step onto the end of a decision-focused system is exactly the pattern the dissertation argues against.

Second, the price of a tradable book. Keeping only the largest K_{card} positions and renormalising, with no retraining, costs a measured 0.08 of Sharpe (Table B.2).

Third, the aggregation correction. Early sweeps averaged per-fold Sharpe ratios; Section 3.6 explains why that is invalid, and Table B.3 shows it is also material: the two aggregations differ by up to 0.48 and can reverse a sign. Every number in this dissertation postdates the correction.

schedule (same model, segment and ledger)	`etf10`	`sector11`
A: the mixture blend	+0.856	+0.332
B: blend restricted to top names	+0.909	+0.342
C: NSGA-II front on the same names	−0.325	−0.231

Table B.1: One trained model, one validation segment, one ledger; only the execution schedule differs.

K_{card}	3	5	7	9	unconstrained
Sharpe	0.546	0.588	0.637	0.663	**0.670**

Table B.2: Exact cardinality on the trained blend, `sector11`, $K = 1$, twelve folds, five seeds, concatenated validation metric. The cost is monotone in tightness.

same configuration, same folds	mean of folds	concatenated
`sector11`, F01–F03, validation	+0.429	−0.053
`etf10`, K=3, test	+0.481	+0.241
`etf10`, single head, test	−0.404	+0.310

Table B.3: An invalid aggregation does not merely shift a number; the last row changes sign.

APPENDIX C

Folds and per-year results

Table C.1 lists every fold. Each fold trains on all data from the universe start (1999 for `sector11`, 2006 for `etf10`) to the end of the year before its validation year. Tables C.2 and C.3 give the per-year consistency breakdown promised in Section 3.6: one excess Sharpe per test year, computed as the headline metric on the seed-averaged daily streams. A single year holds only 52 weekly decisions, so these values are noisy by construction and are read for consistency and not as the headline.

sector11	validation	test
F01	2006	2007
F02	2007	2008
F03	2008	2009
F04	2009	2010
F05	2010	2011
F06	2011	2012
F07	2012	2013
F08	2013	2014
F09	2014	2015
F10	2015	2016
F11	2016	2017
F12	2017	2018
F13	2018	2019
F14	2019	2020
F15	2020	2021
F16	2021	2022
F17	2022	2023
F18	2023	2024

etf10	validation	test
F01	2014	2015
F02	2015	2016
F03	2016	2017
F04	2017	2018
F05	2018	2019
F06	2019	2020
F07	2020	2021
F08	2021	2022
F09	2022	2023
F10	2023	2024

Table C.1: The walk-forward folds. Selection used F01–F08 (`sector11`) and F01–F05 (`etf10`); the held-out verification used F13–F18 and F06–F10.

sector11	1/N	SPY	inv-vol	LW–MV	K=1	K=1+sig
2007	+0.43	+0.14	+0.45	+0.35	+0.44	+0.43
2008	-0.88	-0.85	-0.91	-0.50	-0.96	-0.98
2009	+0.95	+0.90	+0.98	+1.46	+0.95	+0.95
2010	+0.73	+0.70	+0.76	+0.85	+0.68	+0.72
2011	+0.22	+0.15	+0.32	-0.17	+1.34	+1.34
2012	+1.03	+1.07	+1.11	+0.60	+0.52	+0.75
2013	+2.31	+2.41	+2.36	+2.03	+2.16	+2.23
2014	+1.31	+1.29	+1.40	+1.18	+1.29	+1.29
2015	+0.12	+0.27	+0.15	+0.12	+0.13	+0.11
2016	+1.51	+1.40	+1.45	+0.10	+0.92	+1.20
2017	+2.52	+2.64	+2.64	+0.71	+2.41	+2.40
2018	-0.65	-0.46	-0.59	-0.25	-0.54	-0.63
2019	+2.05	+2.03	+2.08	+1.78	+0.37	+1.89
2020	+0.45	+0.64	+0.50	+0.28	+0.41	+0.41
2021	+2.10	+1.93	+2.11	+0.86	+1.78	+1.81
2022	-0.48	-0.72	-0.48	+0.96	-0.54	-0.55
2023	+0.76	+1.38	+0.57	+0.41	+0.74	+0.70
2024	+1.03	+1.46	+0.97	+0.60	+0.98	+0.98

Table C.2: Per-test-year excess Sharpe, `sector11`. K=1 is the frozen arm; the last column adds the five market-level signals.

etf10	1/N	SPY	inv-vol	LW–MV	K=3	K=1+sig
2015	-0.60	+0.27	-0.69	-0.65	-0.40	-0.39
2016	+1.45	+1.40	+1.38	-1.09	+1.18	+1.15
2017	+2.38	+2.64	+2.31	+1.16	+2.29	+2.28
2018	-1.14	-0.46	-1.25	-1.51	-1.11	-1.13
2019	+2.52	+2.03	+2.74	+0.60	+2.45	+2.40
2020	+0.67	+0.64	+0.61	+0.07	+0.59	+0.59
2021	+1.33	+1.93	+1.19	+0.07	+0.42	+0.93
2022	-1.04	-0.72	-1.29	+0.42	-1.02	-1.05
2023	+0.28	+1.38	+0.17	+0.04	+0.24	+0.37
2024	+0.39	+1.46	+0.34	+1.60	+0.21	+0.39

Table C.3: Per-test-year excess Sharpe, `etf10`. K=3 is the frozen arm; the last column is the single head with the five market-level signals.

APPENDIX D

GitLab link

The full project is available at: `https://git.cs.bham.ac.uk/projects-2025-26/hxy511`.

```
dissertation/
  requirements.txt   pinned packages, Python 3.14
  report/            this document (LaTeX)
  regime-rc/         the project
    src/rrc/         library code: data, reservoir, models, optim,
                     backtest, baselines, eval, utils
    scripts/         one entry point per experiment (run_*.py)
                     and per figure (make_*_figures.py)
    configs/         YAML configurations; configs/frozen/ holds
                     the frozen defaults
    tests/           pytest suite (make test)
    results/         append-only registry.csv, runs and figures
    Makefile         every pipeline step as a copy-paste recipe
```

There are two extra folders seen in gitlab as well being portfolio-data and portfolio-forecast, they were done as initial testing and not important for this report. To run: create a virtual environment, install `requirements.txt`, and work from inside `regime-rc/`. `make data` builds the data artifacts (cached locally, offline after the first pull), `make test` runs the test suite, and the remaining targets in the `Makefile` map one-to-one onto the experiments of Chapter 5. Every `run_*.py` script requires `--universe etf10` or `--universe sector11`. The one exception to re-running is `scripts/run_holdout.py`, which was run exactly once by design and should not be run again.